2026–2027 PROFESSIONAL EDITION (VERSION 2.0)

Volume II: The CKA Practical Workbook

67 Hands-On Production Labs with Automated Verifications & Solutions

Cloud Native Engineering Press

Targeting Kubernetes v1.31.x / v1.32.x (Forward Compatible through v1.33+)

Independent Professional Certification Study System

Certified Kubernetes Administrator (CNCF / The Linux Foundation)

VOLUME II: THE CKA PRACTICAL WORKBOOK

DOMAIN 1: CLUSTER ARCHITECTURE, INSTALLATION & CONFIGURATION (LABS 01–15)

LAB 01: Multi-Namespace RBAC Role and RoleBinding Deployment

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Author a namespace-scoped `Role` and bind it to a `ServiceAccount` to permit granular pod management while denying service modifications.
- **Prerequisites:** Running cluster with `kubectl` configured.
- **Cluster Environment:** Any standard Kubernetes cluster (`kind`, `minikube`, or `kubeadm`).
- **Initial State:** Namespace `finance` does not exist.
- **Task Description:**
    1. Create a namespace named `finance`.
    2. Create a `ServiceAccount` named `finance-auditor` in namespace `finance`.
    3. Create a `Role` named `pod-reader` in namespace `finance` that allows `get`, `list`, and `watch` verbs on `pods` and `pods/log`.
    4. Create a `RoleBinding` named `bind-pod-reader` binding the `finance-auditor` ServiceAccount to the `pod-reader` Role.
- **Constraints:** Do not grant any permissions outside of namespace `finance`. Do not grant access to `secrets` or `services`.
- **Hints:** Use `kubectl create role` and `kubectl create rolebinding` with `--dry-run=client -o yaml` for fast execution.
- **Expected Result:** The `finance-auditor` ServiceAccount can list pods in `finance`, but cannot list pods in `default` or create deployments.
- **Verification:**

```
kubectl auth can-i list pods -n finance --as=system:serviceaccount:finance:finance-auditor
# Expected: yes
kubectl auth can-i create pods -n finance --as=system:serviceaccount:finance:finance-auditor
# Expected: no
kubectl auth can-i list pods -n default --as=system:serviceaccount:finance:finance-auditor
# Expected: no
```

- **Full Solution:**

```
kubectl create namespace finance
kubectl create serviceaccount finance-auditor -n finance
kubectl create role pod-reader -n finance --verb=get,list,watch --resource=pods,pods/log
kubectl create rolebinding bind-pod-reader -n finance --role=pod-reader --serviceaccount=finance:fi
```

- **Common Mistakes:** Omitting `-n finance` on the Role or RoleBinding, creating them in `default`.
- **Troubleshooting:** If `auth can-i` returns `no`, inspect `kubectl get rolebinding bind-pod-reader -n finance -o yaml` to check the `subjects` namespace.
- **Cleanup Instructions:**

```
kubectl delete namespace finance
```

LAB 02: Cluster-Wide Privilege Escalation Prevention via ClusterRoles

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Create a `ClusterRole` and bind it with a namespace-scoped `RoleBinding` to constrain cluster-wide role definitions to a single namespace.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Multi-namespace cluster.
- **Initial State:** Namespace `engineering` exists.
- **Task Description:**
    1. Create a `ClusterRole` named `node-and-deployment-viewer` granting `get`, `list`, and `watch` on `nodes` and `deployments`.
    2. Create a ServiceAccount named `eng-lead` in namespace `engineering`.
    3. Bind the `ClusterRole` to `eng-lead` in namespace `engineering` using a `RoleBinding` named `eng-lead-deploy-view`.
- **Constraints:** Do NOT use a `ClusterRoleBinding`.
- **Hints:** A `RoleBinding` can reference a `ClusterRole`, but it only grants permissions on namespaced resources within its own namespace.

- **Expected Result:** `eng-lead` can list deployments in `engineering`, but CANNOT list nodes or list deployments in `default`.
- **Verification:**

```
kubectl auth can-i list deployments -n engineering --as=system:serviceaccount:engineering:eng-lead
# Expected: yes
kubectl auth can-i list deployments -n default --as=system:serviceaccount:engineering:eng-lead
# Expected: no
kubectl auth can-i list nodes --as=system:serviceaccount:engineering:eng-lead
# Expected: no (nodes are cluster-scoped, RoleBinding cannot grant them)
```

- **Full Solution:**

```
kubectl create namespace engineering --dry-run=client -o yaml | kubectl apply -f -
kubectl create clusterrole node-and-deployment-viewer --verb=get,list,watch --resource=nodes,deploy
kubectl create serviceaccount eng-lead -n engineering
kubectl create rolebinding eng-lead-deploy-view -n engineering --clusterrole=node-and-deployment-vi
```

- **Common Mistakes:** Attempting to grant node permissions via a `RoleBinding`. Non-namespaced resources (like Nodes) require a `ClusterRoleBinding`.
- **Troubleshooting:** Run `kubectl describe rolebinding eng-lead-deploy-view -n engineering` and verify `RoleRef` points to `ClusterRole/node-and-deployment-viewer`.
- **Cleanup Instructions:**

```
kubectl delete namespace engineering
kubectl delete clusterrole node-and-deployment-viewer
```

LAB 03: Safe Worker Node Maintenance (Cordon and Drain)

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Drain all non-daemon workloads from a worker node, perform mock OS patching, and safely return the node to active service.
- **Prerequisites:** Multi-node cluster (1 control plane, at least 2 worker nodes).
- **Cluster Environment:** `kubeadm` multi-node cluster or multi-node `kind`.
- **Initial State:** Deployment `nginx-fleet` with 4 replicas distributed across worker nodes.
- **Task Description:**
    1. Identify worker node `worker-1`.
    2. Safely cordon `worker-1` to prevent new Pod scheduling.
    3. Drain all workloads from `worker-1`, evicting pods cleanly while ignoring DaemonSets and handling local emptyDir volumes.

4. Verify all 4 replicas of `nginx-fleet` are rescheduled onto other nodes.
5. Uncordon `worker-1`.

- **Constraints:** No downtime for `nginx-fleet`.
- **Hints:** Use `--ignore-daemonsets` and `--delete-emptydir-data`.
- **Expected Result:** Node `worker-1` enters `SchedulingDisabled`, all pods migrate to other nodes, and then node returns to `Ready`.
- **Verification:**

```
kubectl get nodes worker-1
# During drain: SchedulingDisabled
kubectl get pods -l app=nginx-fleet -o wide
# None should reside on worker-1 during maintenance
```

- **Full Solution:**

```
kubectl create deployment nginx-fleet --image=nginx:1.27 --replicas=4
kubectl cordon worker-1
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data --force
# Verify node is evacuated
kubectl get pods -A -o wide | grep worker-1
# Uncordon
kubectl uncordon worker-1
```

- **Common Mistakes:** Forgetting `--ignore-daemonsets`, causing the drain command to fail immediately.
- **Troubleshooting:** If drain hangs, check for a `PodDisruptionBudget` blocking eviction.
- **Cleanup Instructions:**

```
kubectl delete deployment nginx-fleet
```

LAB 04: Control Plane Version Upgrade with kubeadm

- **Difficulty:** Advanced
- **Estimated Duration:** 15 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Execute an in-place minor version upgrade of a Kubernetes control plane node using `kubeadm`, `kubelet`, and `kubectl`.
- **Prerequisites:** SSH access to control plane node with `sudo`.
- **Cluster Environment:** `kubeadm` cluster running v1.30.x.
- **Initial State:** Cluster running v1.30.2.
- **Task Description:**

1. Upgrade `kubeadm` package to `1.31.2-1.1`.
2. Plan and execute `kubeadm upgrade apply v1.31.2`.

3. Drain the control plane node.
4. Upgrade `kubelet` and `kubectl` to `1.31.2-1.1`.
5. Restart `kubelet` and uncordon the control plane.

- **Constraints:** Do not skip minor versions. Do not reboot the machine.
- **Hints:** Remember to unhold packages before `apt-get install` and hold them immediately afterward.
- **Expected Result:** `kubectl get nodes` reports the control plane running `v1.31.2`.
- **Verification:**

```
kubectl get nodes
kubeadm version
```

- **Full Solution:**

```
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm

sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.31.2 -y

kubectl drain $(hostname) --ignore-daemonsets --delete-emptydir-data

sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl

sudo systemctl daemon-reload
sudo systemctl restart kubelet

kubectl uncordon $(hostname)
```

- **Common Mistakes:** Upgrading `kubelet` before running `kubeadm upgrade apply`.
- **Troubleshooting:** Check `journalctl -u kubelet -e` if kubelet fails to start after version upgrade.
- **Cleanup Instructions:** None (cluster remains upgraded).

LAB 05: Worker Node Upgrade with kubeadm

- **Difficulty:** Advanced
- **Estimated Duration:** 12 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Upgrade a secondary worker node to match the upgraded control plane version.
- **Prerequisites:** Control plane already upgraded to v1.31.2; SSH access to `worker-1`.
- **Cluster Environment:** Multi-node `kubeadm` cluster.
- **Initial State:** `worker-1` is at v1.30.2 while control plane is at v1.31.2.
- **Task Description:**

1. From control plane, drain `worker-1`.
2. SSH into `worker-1`.
3. Upgrade `kubeadm` to `1.31.2-1.1`.
4. Execute `sudo kubeadm upgrade node`.
5. Upgrade `kubelet` and `kubectl` to `1.31.2-1.1`.
6. Restart `kubelet`.
7. From control plane, uncordon `worker-1`.

- **Constraints:** Do not run `kubeadm upgrade apply` on a worker node.
- **Hints:** Use `kubeadm upgrade node` on workers.
- **Expected Result:** `kubectl get nodes` shows `worker-1` running `v1.31.2` in `Ready` status.
- **Verification:**

```
kubectl get node worker-1 -o custom-columns=NAME:.metadata.name,VERSION:.status.nodeInfo.kubeletVer
```

- **Full Solution:**

```
# On Control Plane:
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# On worker-1:
sudo apt-mark unhold kubeadm
sudo apt-get update && sudo apt-get install -y kubeadm=1.31.2-1.1
sudo apt-mark hold kubeadm

sudo kubeadm upgrade node

sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo apt-mark hold kubelet kubectl

sudo systemctl daemon-reload
sudo systemctl restart kubelet
exit

# On Control Plane:
kubectl uncordon worker-1
```

- **Common Mistakes:** Running `kubeadm upgrade apply` on worker nodes instead of `kubeadm upgrade node`.
- **Troubleshooting:** If the node stays in `SchedulingDisabled`, check if `kubectl uncordon` was run on the control plane.
- **Cleanup Instructions:** None.

LAB 06: etcd Automated Snapshot Backup with mTLS

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Use `etcdctl` to capture a verified cryptographic snapshot of the cluster state.
- **Prerequisites:** SSH access to control plane node with root/sudo access.
- **Cluster Environment:** Single or multi-control-plane cluster with static pod etcd.
- **Initial State:** Healthy etcd cluster listening on port 2379.
- **Task Description:**
    1. Identify etcd client certificates in `/etc/kubernetes/pki/etcd/`.
    2. Export `ETCDCTL_API=3`.
    3. Save a snapshot to `/var/backups/etcd-snapshot-manual.db`.
    4. Verify the snapshot status and record its total keys and revision number.
- **Constraints:** Must use TLS certificates: `--cacert`, `--cert`, `--key`.
- **Hints:** Pass `--endpoints=https://127.0.0.1:2379`.
- **Expected Result:** Snapshot file exists at `/var/backups/etcd-snapshot-manual.db` with status showing valid hash.
- **Verification:**

```
sudo ETCDCTL_API=3 etcdctl --write-out=table snapshot status /var/backups/etcd-snapshot-manual.db
```

- **Full Solution:**

```
sudo mkdir -p /var/backups
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /var/backups/etcd-snapshot-manual.db
```

- **Common Mistakes:** Forgetting `ETCDCTL_API=3`, causing the command to default to API v2 and fail.
- **Troubleshooting:** If `connection refused`, verify port 2379 is listening with `netstat -tlpn | grep 2379`.
- **Cleanup Instructions:** None (retain backup for Lab 07).

LAB 07: Disaster Recovery: etcd Snapshot Restoration

- **Difficulty:** Advanced

- **Estimated Duration:** 15 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Restore a previously captured etcd snapshot into a new data directory and update static pod manifests.
- **Prerequisites:** Completed Lab 06 (snapshot at `/var/backups/etcd-snapshot-manual.db`).
- **Cluster Environment:** `kubeadm` control plane node.
- **Initial State:** Cluster running with data created after the snapshot.
- **Task Description:**
    1. Create a canary deployment `pre-restore-canary` (will be deleted by restore).
    2. Stop the API Server by moving `/etc/kubernetes/manifests/kube-apiserver.yaml` to `/tmp/`.
    3. Restore `/var/backups/etcd-snapshot-manual.db` into new directory `/var/lib/etcd-from-backup`.
    4. Update `/etc/kubernetes/manifests/etcd.yaml` to use `/var/lib/etcd-from-backup` for hostPath volume mounts.
    5. Move `kube-apiserver.yaml` back to `/etc/kubernetes/manifests/`.
    6. Verify cluster state is restored and `pre-restore-canary` is gone.
- **Constraints:** Do not overwrite existing `/var/lib/etcd` directory directly.
- **Hints:** Modify both `volumeMounts` and `volumes` in `etcd.yaml`.
- **Expected Result:** API server restarts, etcd mounts new directory, canary deployment no longer exists.
- **Verification:**

```
kubectl get deployment pre-restore-canary
# Expected: Error from server (NotFound)
kubectl get nodes
# Expected: Ready
```

- **Full Solution:**

```
kubectl create deployment pre-restore-canary --image=nginx:alpine

# 1. Stop static pods
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/
sudo mv /etc/kubernetes/manifests/etcd.yaml /tmp/

# 2. Restore to new dir
sudo ETCDCTL_API=3 etcdctl \
  --data-dir=/var/lib/etcd-from-backup \
  snapshot restore /var/backups/etcd-snapshot-manual.db

# 3. Update etcd manifest
sudo sed -i 's|/var/lib/etcd|/var/lib/etcd-from-backup|g' /tmp/etcd.yaml

# 4. Restore manifests
sudo mv /tmp/etcd.yaml /etc/kubernetes/manifests/
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/

# 5. Wait for API Server
sleep 15
kubectl get nodes
```

- **Common Mistakes:** Overwriting `/var/lib/etcd` without stopping the static pod, causing write collisions.
- **Troubleshooting:** Use `crictl ps -a | grep etcd` to check if the restored etcd container launched successfully.
- **Cleanup Instructions:**

```
sudo rm -rf /var/lib/etcd-from-backup
# (Optionally revert etcd.yaml back to /var/lib/etcd)
```

LAB 08: Deploying Applications via Helm with Values Overrides

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Install and configure an application using Helm CLI, pass custom parameters via `--set`, and verify release lifecycle.
- **Prerequisites:** Helm v3 installed on candidate workstation.
- **Cluster Environment:** Standard Kubernetes cluster.
- **Initial State:** Namespace `monitoring-tools` does not exist.
- **Task Description:**
  1. Add the Prometheus community repo: `https://prometheus-community.github.io/helm-charts`.
  2. Install a release named `node-metrics` using chart `prometheus-community/prometheus-node-exporter`.
  3. Deploy into namespace `monitoring-tools` (create if needed).

4. Set the hostNetwork configuration to `true` and service port to `9100`.
5. List the installed releases and verify status is `deployed`.

- **Constraints:** Use Helm CLI only; do not write raw deployment manifests.
- **Hints:** Use `--namespace <ns> --create-namespace`.
- **Expected Result:** Helm release `node-metrics` is deployed and running DaemonSet pods on every node.
- **Verification:**

```
helm list -n monitoring-tools
kubectl get daemonset -n monitoring-tools
```

- **Full Solution:**

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install node-metrics prometheus-community/prometheus-node-exporter \
  --namespace monitoring-tools \
  --create-namespace \
  --set service.port=9100
```

- **Common Mistakes:** Typing the repo name incorrectly or forgetting `helm repo update`.
- **Troubleshooting:** Run `helm status node-metrics -n monitoring-tools` to view deployment notes and error states.
- **Cleanup Instructions:**

```
helm uninstall node-metrics -n monitoring-tools
kubectl delete namespace monitoring-tools
```

LAB 09: Declarative Multi-Environment Overlays with Kustomize

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Structure a base directory and production overlay using Kustomize, and apply it directly using `kubectl apply -k`.
- **Prerequisites:** `kubectl` with native Kustomize support.
- **Cluster Environment:** Any test cluster.
- **Initial State:** Clean working directory `/tmp/kustomize-lab`.
- **Task Description:**

1. Create directory structure `/tmp/kustomize-lab/base` and `/tmp/kustomize-lab/overlays/prod`.
2. In `base`, author a Deployment `app-server` running `nginx:1.27` with 1 replica and a `kustomization.yaml`.

3. In `overlays/prod`, author a `kustomization.yaml` that:
    - Pulls from `../../base`.
    - Adds name prefix `prod-`.
    - Sets common labels `env: production` and `tier: gold`.
    - Patches replica count to `3`.
4. Apply the production overlay using `kubectl apply -k`.

- **Constraints:** Do not modify files in `base` when creating the production overlay.
- **Hints:** Use `kubectl kustomize` to preview the rendered YAML before applying.
- **Expected Result:** A Deployment named `prod-app-server` with 3 replicas is created.
- **Verification:**

```
kubectl get deployment prod-app-server -o jsonpath='{.spec.replicas}'
# Expected: 3
kubectl get pods -l env=production,tier=gold
```

- **Full Solution:**

```
mkdir -p /tmp/kustomize-lab/base /tmp/kustomize-lab/overlays/prod

# Base
cat <<EOF > /tmp/kustomize-lab/base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-server
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app-server
  template:
    metadata:
      labels:
        app: app-server
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
EOF

cat <<EOF > /tmp/kustomize-lab/base/kustomization.yaml
resources:
- deployment.yaml
EOF

# Overlay
cat <<EOF > /tmp/kustomize-lab/overlays/prod/replica-patch.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-server
spec:
  replicas: 3
EOF

cat <<EOF > /tmp/kustomize-lab/overlays/prod/kustomization.yaml
resources:
- ../../base
namePrefix: prod-
commonLabels:
  env: production
  tier: gold
patches:
- path: replica-patch.yaml
EOF

kubectl apply -k /tmp/kustomize-lab/overlays/prod
```

- **Common Mistakes:** Misspelling `resources` as `resource` in `kustomization.yaml`.
- **Troubleshooting:** Run `kubectl kustomize /tmp/kustomize-lab/overlays/prod` to view exact schema validation errors.
- **Cleanup Instructions:**

```
kubectl delete -k /tmp/kustomize-lab/overlays/prod
rm -rf /tmp/kustomize-lab
```

LAB 10: Custom Resource Definition (CRD) Inspection and Instantiation

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Install a Custom Resource Definition, inspect its schema with `kubectl explain`, and instantiate a Custom Resource.
- **Prerequisites:** Cluster administrator privileges.
- **Cluster Environment:** Any test cluster.
- **Initial State:** No CRD for `Database` exists.
- **Task Description:**
    1. Author and apply a CRD `databases.example.com` with group `example.com`, version `v1`, kind `Database`, and scope `Namespaced`. Include a string spec field `engine` and integer field `replicas`.
    2. Use `kubectl explain database.spec` to verify the schema.
    3. Create a Custom Resource of kind `Database` named `orders-db` with `engine: postgres` and `replicas: 2`.
- **Constraints:** Schema must be validated OpenAPI v3.
- **Hints:** Root API group for CRD definitions is `apiextensions.k8s.io/v1`.
- **Expected Result:** `kubectl get databases` lists `orders-db`.
- **Verification:**

```
kubectl get crd databases.example.com
kubectl get databases orders-db -o yaml
```

- **Full Solution:**

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.example.com
spec:
  group: example.com
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              engine:
                type: string
              replicas:
                type: integer
  scope: Namespaced
  names:
    plural: databases
    singular: database
    kind: Database
    shortNames:
    - db
---
apiVersion: example.com/v1
kind: Database
metadata:
  name: orders-db
spec:
  engine: postgres
  replicas: 2
```

- **Common Mistakes:** Omitting `plural` in `names`, which causes CRD validation rejection.
- **Troubleshooting:** If `kubectl explain` fails, check if the CRD version has `served: true`.
- **Cleanup Instructions:**

```
kubectl delete crd databases.example.com
```

LAB 11: ServiceAccount Token Projection & Expiration Tuning

- **Difficulty:** Intermediate
- **Estimated Duration:** 8 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Configure a Pod with projected time-bound ServiceAccount tokens and custom audience claims.

- **Prerequisites:** Cluster running v1.24+.
- **Cluster Environment:** Standard Kubernetes cluster.
- **Initial State:** ServiceAccount `vault-auth-sa` exists.
- **Task Description:**
    1. Create a ServiceAccount named `vault-auth-sa` in `default`.
    2. Create a Pod `vault-client` running `alpine` that mounts a projected ServiceAccount token at `/var/run/secrets/tokens/vault-token`.
    3. Configure token expiration to 7200 seconds and audience to `vault.hashicorp.internal`.
    4. Inspect the token from inside the running Pod container.
- **Constraints:** Do not use legacy static Secret tokens.
- **Hints:** Use `spec.volumes[*].projected.sources[*].serviceAccountToken`.
- **Expected Result:** The token file exists inside the container and can be read.
- **Verification:**

```
kubectl exec vault-client -- ls -la /var/run/secrets/tokens/vault-token
```

- **Full Solution:**

```
kubectl create serviceaccount vault-auth-sa
```

```
apiVersion: v1
kind: Pod
metadata:
  name: vault-client
spec:
  serviceAccountName: vault-auth-sa
  containers:
  - name: client
    image: alpine
    command: ["sleep", "3600"]
    volumeMounts:
    - mountPath: /var/run/secrets/tokens
      name: vault-token-vol
  volumes:
  - name: vault-token-vol
    projected:
      sources:
      - serviceAccountToken:
          path: vault-token
          expirationSeconds: 7200
          audience: vault.hashicorp.internal
```

- **Common Mistakes:** Omitting `expirationSeconds` or setting it below 600 seconds (minimum is 10 minutes).
- **Troubleshooting:** If container fails with mount error, check Pod events with `kubectl describe pod vault-client`.
- **Cleanup Instructions:**

```
kubectl delete pod vault-client
kubectl delete serviceaccount vault-auth-sa
```

LAB 12: Certificate Expiration Audit and Manual Renewal

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Inspect expiration dates of all control plane TLS certificates and execute targeted renewals using `kubeadm`.
- **Prerequisites:** SSH access to control plane node with sudo privileges.
- **Cluster Environment:** `kubeadm` single or multi-master cluster.
- **Initial State:** Cluster certificates in `/etc/kubernetes/pki/`.
- **Task Description:**
    1. Inspect the expiration of all certificates on the control plane node.
    2. Identify the expiration date of `apiserver-kubelet-client.crt`.
    3. Manually renew strictly the `apiserver-kubelet-client` certificate without renewing all certs.
    4. Verify the renewed expiration date has moved 1 year into the future.
- **Constraints:** Do not break API server connectivity.
- **Hints:** Use `kubeadm certs check-expiration` and `kubeadm certs renew <cert-name>`.
- **Expected Result:** `apiserver-kubelet-client` certificate shows renewed 1-year validity.
- **Verification:**

```
sudo kubeadm certs check-expiration | grep apiserver-kubelet-client
```

- **Full Solution:**

```
# Check expiration
sudo kubeadm certs check-expiration

# Renew single cert
sudo kubeadm certs renew apiserver-kubelet-client

# Verify
sudo kubeadm certs check-expiration | grep apiserver-kubelet-client
```

- **Common Mistakes:** Running `kubeadm certs renew all` when the task explicitly asked for a single certificate renewal.
- **Troubleshooting:** If the certificate does not update, ensure the command was executed with `sudo`.
- **Cleanup Instructions:** None.

LAB 13: High Availability Control Plane Sizing and Quorum Calculation

- **Difficulty:** Intermediate
- **Estimated Duration:** 8 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Calculate exact etcd quorum boundaries, fault tolerances, and control plane load balancer requirements for production HA clusters.
- **Prerequisites:** Conceptual understanding of Raft consensus.
- **Cluster Environment:** Paper / CLI scenario.
- **Initial State:** Cluster sizing inquiry.
- **Task Description:**
    1. Create a script or manifest documenting the sizing requirements for an HA cluster designed to survive the simultaneous failure of 2 etcd nodes.
    2. Determine minimum etcd node count, quorum number, and failure tolerance.
    3. Author a ConfigMap named `ha-cluster-specs` in `kube-system` containing keys: `etcd_nodes`, `quorum_needed`, `tolerated_failures`.
- **Constraints:** Quorum formula $\lfloor N/2 \rfloor + 1$ must be strictly adhered to.
- **Hints:** To survive 2 failures, $N=5$, because Quorum $= \lfloor 5/2 \rfloor + 1 = 3$. ($5 - 2 = 3$).
- **Expected Result:** ConfigMap reflects accurate distributed consensus topology.
- **Verification:**

```
kubectl get configmap ha-cluster-specs -n kube-system -o yaml
```

- **Full Solution:**

```
kubectl create configmap ha-cluster-specs -n kube-system \
  --from-literal=etcd_nodes=5 \
  --from-literal=quorum_needed=3 \
  --from-literal=tolerated_failures=2
```

- **Common Mistakes:** Assuming a 4-node etcd cluster provides more fault tolerance than a 3-node cluster (both tolerate only 1 failure, but 4-node has higher risk of split-brain).
- **Troubleshooting:** Recheck Raft odd-number quorum rules.
- **Cleanup Instructions:**

```
kubectl delete configmap ha-cluster-specs -n kube-system
```

LAB 14: Containerd Socket & Cgroup Driver Alignment

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Verify and enforce `SystemdCgroup = true` in `/etc/containerd/config.toml` to prevent kubelet cgroup panic.
- **Prerequisites:** SSH access to a worker node.
- **Cluster Environment:** Node running containerd.
- **Initial State:** Default or unverified containerd configuration.
- **Task Description:**
    1. Inspect `/etc/containerd/config.toml` on `worker-1`.
    2. Ensure `SystemdCgroup` is set to `true` under `runc.options`.
    3. Restart `containerd` and `kubelet`.
    4. Verify using `crictl info` that the runtime endpoint is active and reports healthy.
- **Constraints:** Do not corrupt the TOML syntax.
- **Hints:** If `/etc/containerd/config.toml` is empty or missing, generate defaults with `containerd config default`.
- **Expected Result:** `containerd` and `kubelet` running cleanly with systemd cgroups.
- **Verification:**

```
sudo crictl info | jq .status.conditions
sudo systemctl is-active kubelet
# Expected: active
```

- **Full Solution:**

```
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl restart kubelet
```

- **Common Mistakes:** Editing the wrong `[plugins."io.containerd.grpc.v1.cri"...]` block.
- **Troubleshooting:** If containerd fails to restart, run `sudo containerd config dump` to find the exact line with the TOML syntax error.
- **Cleanup Instructions:** None.

LAB 15: Static Pod Provisioning on Specific Worker Node

- **Difficulty:** Intermediate

- **Estimated Duration:** 10 minutes
- **Domain:** Cluster Architecture, Installation & Configuration (25%)
- **Learning Objective:** Provision a static Pod on a worker node by directly placing a manifest in the kubelet's static pod path.
- **Prerequisites:** SSH access to `worker-1`.
- **Cluster Environment:** Multi-node cluster.
- **Initial State:** Kubelet running on `worker-1`.
- **Task Description:**
    1. Inspect `/var/lib/kubelet/config.yaml` on `worker-1` to locate `staticPodPath`.
    2. If `staticPodPath` is not defined, add `staticPodPath: /etc/kubernetes/manifests` and restart `kubelet`.
    3. Place a Pod manifest `worker-monitor.yaml` in `/etc/kubernetes/manifests/` running `busybox:1.36` sleeping 3600.
    4. On the control plane, verify the mirror pod `worker-monitor-worker-1` appears in `kubectl get pods`.
- **Constraints:** Do not use `kubectl create` or `kubectl apply` to launch the Pod.
- **Hints:** Static pods are named `<pod-name>-<node-name>`.
- **Expected Result:** Mirror pod is visible to the API server in `Running` status.
- **Verification:**

```
kubectl get pods -A | grep worker-monitor
```

- **Full Solution:**

```
# On worker-1:
sudo mkdir -p /etc/kubernetes/manifests
# Verify /var/lib/kubelet/config.yaml has staticPodPath: /etc/kubernetes/manifests
cat <<EOF | sudo tee /etc/kubernetes/manifests/worker-monitor.yaml
apiVersion: v1
kind: Pod
metadata:
  name: worker-monitor
spec:
  containers:
  - name: monitor
    image: busybox:1.36
    command: ["sleep", "3600"]
EOF

# If needed, restart kubelet:
sudo systemctl restart kubelet

# On Control Plane:
kubectl get pods | grep worker-monitor
```

- **Common Mistakes:** Attempting to delete the mirror pod via `kubectl delete pod`; it will immediately be recreated by the worker's kubelet until the file is deleted from disk!
- **Troubleshooting:** If mirror pod does not appear, check `journalctl -u kubelet -e` on `worker-1`.

- **Cleanup Instructions:**

```
# On worker-1:
sudo rm -f /etc/kubernetes/manifests/worker-monitor.yaml
```

VOLUME II: THE CKA PRACTICAL WORKBOOK

DOMAIN 2: WORKLOADS & SCHEDULING (LABS 16–30)

LAB 16: Zero-Downtime Deployment Rolling Updates and Revisions

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Orchestrate an application rolling update, inspect rollout history, and execute a surgical rollback to a specific historical revision.
- **Prerequisites:** Running cluster.
- **Cluster Environment:** Standard Kubernetes cluster.
- **Initial State:** Clean namespace `web-workloads`.
- **Task Description:**
    1. Create namespace `web-workloads`.
    2. Deploy `nginx-deployment` with 3 replicas running image `nginx:1.25` in `web-workloads`.
    3. Update image to `nginx:1.26` using `kubectl set image`, recording the rollout command.
    4. Update image to a non-existent tag `nginx:1.999-broken` to simulate a broken release.
    5. Check rollout status and observe the failure.
    6. Inspect revision history and roll back to the stable revision running `nginx:1.26`.
- **Constraints:** Keep at least 2 replicas available at all times during rollout.
- **Hints:** Use `kubectl rollout history` and `kubectl rollout undo --to-revision=<N>`.
- **Expected Result:** Deployment stabilizes at 3 replicas running `nginx:1.26`.
- **Verification:**

```
kubectl get deployment nginx-deployment -n web-workloads -o jsonpath='{.spec.template.spec.containe
# Expected: nginx:1.26
```

- **Full Solution:**

```
kubectl create namespace web-workloads
kubectl create deployment nginx-deployment --image=nginx:1.25 --replicas=3 -n web-workloads
kubectl set image deployment/nginx-deployment nginx=nginx:1.26 -n web-workloads
kubectl rollout status deployment/nginx-deployment -n web-workloads
kubectl set image deployment/nginx-deployment nginx=nginx:1.999-broken -n web-workloads
# Observe failure
kubectl rollout undo deployment/nginx-deployment --to-revision=2 -n web-workloads
```

- **Common Mistakes:** Rolling back without checking revision numbers, reverting to the wrong version.
- **Troubleshooting:** Run `kubectl describe deployment nginx-deployment -n web-workloads` to view condition progress.
- **Cleanup Instructions:**

```
kubectl delete namespace web-workloads
```

LAB 17: StatefulSet Deployment with Sequential Pod Management

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Deploy a StatefulSet paired with a headless Service and observe ordered creation and stable network hostnames.
- **Prerequisites:** Cluster with default StorageClass.
- **Cluster Environment:** Any Kubernetes cluster.
- **Initial State:** Namespace `stateful-apps`.
- **Task Description:**
  1. Create namespace `stateful-apps`.
  2. Create a headless Service `db-cluster-svc` with `clusterIP: None` exposing port 27017 for selector `app: mongo`.
  3. Create a StatefulSet `mongo` with 3 replicas, serviceName `db-cluster-svc`, and image `mongo:6.0`.
  4. Verify pods launch sequentially: `mongo-0`, then `mongo-1`, then `mongo-2`.
  5. Scale the StatefulSet to 4 replicas and verify ordered scale-up.
- **Constraints:** Must use headless Service for network identity.
- **Hints:** In a StatefulSet, `mongo-1` will not start until `mongo-0` is in `Running` and ready.
- **Expected Result:** 4 sequential pods running with predictable names.
- **Verification:**

```
kubectl get pods -n stateful-apps -l app=mongo
```

- **Full Solution:**

```
apiVersion: v1
kind: Namespace
metadata:
  name: stateful-apps
---
apiVersion: v1
kind: Service
metadata:
  name: db-cluster-svc
  namespace: stateful-apps
spec:
  clusterIP: None
  selector:
    app: mongo
  ports:
  - port: 27017
    name: mongo
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mongo
  namespace: stateful-apps
spec:
  serviceName: db-cluster-svc
  replicas: 3
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
      - name: mongo
        image: mongo:6.0
        ports:
        - containerPort: 27017
```

```
kubectl apply -f statefulset.yaml
kubectl scale statefulset mongo --replicas=4 -n stateful-apps
```

- **Common Mistakes:** Omitting `serviceName` in the StatefulSet spec.
- **Troubleshooting:** If pods do not advance to the next index, inspect the previous pod's readiness probe.
- **Cleanup Instructions:**

```
kubectl delete namespace stateful-apps
```

LAB 18: DaemonSet Node Coverage and Host Resource Access

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Deploy a DaemonSet targeting all worker nodes while tolerating control plane taints to collect host system metrics.
- **Prerequisites:** Multi-node cluster with tainted control plane.
- **Cluster Environment:** `kubeadm` or multi-node `kind`.
- **Initial State:** Namespace `monitoring`.
- **Task Description:**
  1. Create namespace `monitoring`.
  2. Create a DaemonSet named `host-monitor` in namespace `monitoring`.
  3. Run container image `busybox:1.36` executing `sleep 86400`.
  4. Add a toleration so the DaemonSet pod also schedules on the control plane node: `key: node-role.kubernetes.io/control-plane, effect: NoSchedule`.
  5. Verify that the number of running pods exactly matches the total node count (workers + control plane).
- **Constraints:** Must use DaemonSet, not Deployment.
- **Hints:** Check node count with `kubectl get nodes --no-headers | wc -l`.
- **Expected Result:** Exactly 1 pod runs on every node in the cluster.
- **Verification:**

```
kubectl get ds host-monitor -n monitoring
# Desired should equal Current and Ready
```

- **Full Solution:**

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: host-monitor
  namespace: monitoring
spec:
  selector:
    matchLabels:
      app: host-monitor
  template:
    metadata:
      labels:
        app: host-monitor
    spec:
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        operator: Exists
        effect: NoSchedule
      containers:
      - name: monitor
        image: busybox:1.36
        command: ["sleep", "86400"]
```

```
kubectl create namespace monitoring
kubectl apply -f daemonset.yaml
```

- **Common Mistakes:** Omitting the control plane toleration, resulting in pods only scheduling on worker nodes.
- **Troubleshooting:** Run `kubectl get pods -n monitoring -o wide` to verify each node holds one pod.
- **Cleanup Instructions:**

```
kubectl delete namespace monitoring
```

LAB 19: Batch Jobs with Parallelism and Completion Quotas

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Configure a Kubernetes Job that requires 6 total completions with up to 2 pods running concurrently.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `batch-jobs`.
- **Task Description:**
  1. Create namespace `batch-jobs`.
  2. Create a Job named `data-processor` in namespace `batch-jobs`.

3. Image: `busybox:1.36`. Command: `sh -c "echo processing partition; sleep 3"`.
4. Set `completions: 6` and `parallelism: 2`.
5. Set `backoffLimit: 3`.
6. Monitor the Job until status reaches `Completed`.

- **Constraints:** Ensure `restartPolicy: OnFailure` or `Never`.
- **Hints:** Use `kubectl create job data-processor --image=busybox:1.36 --dry-run=client -o yaml` and edit `completions` and `parallelism`.
- **Expected Result:** Job finishes with `6/6` completions.
- **Verification:**

```
kubectl get job data-processor -n batch-jobs
# COMPLETIONS: 6/6
```

- **Full Solution:**

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-processor
  namespace: batch-jobs
spec:
  completions: 6
  parallelism: 2
  backoffLimit: 3
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: processor
        image: busybox:1.36
        command: ["/bin/sh", "-c", "echo processing partition; sleep 3"]
```

```
kubectl create namespace batch-jobs
kubectl apply -f job.yaml
kubectl get jobs -n batch-jobs -w
```

- **Common Mistakes:** Setting `restartPolicy: Always`, which is illegal in Job Pod templates.
- **Troubleshooting:** Run `kubectl describe job data-processor -n batch-jobs` to view active pods and completion timestamps.
- **Cleanup Instructions:**

```
kubectl delete namespace batch-jobs
```

LAB 20: CronJob with Concurrency Policies

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Schedule a recurring task using CronJob and enforce `concurrencyPolicy: Forbid` to prevent overlapping runs.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `cron-jobs`.
- **Task Description:**
    1. Create namespace `cron-jobs`.
    2. Create a CronJob `log-cleaner` running every minute (`* * * * *`).
    3. Image: `busybox:1.36`. Command: `sh -c "echo Cleaning logs; sleep 70"`.
    4. Set `concurrencyPolicy: Forbid`.
    5. Set `successfulJobsHistoryLimit: 2` and `failedJobsHistoryLimit: 1`.
    6. Observe that because the job sleeps for 70 seconds, the 1-minute schedule skips the second iteration rather than running concurrent pods.
- **Constraints:** Must use `batch/v1`.
- **Hints:** `concurrencyPolicy` options are `Allow`, `Forbid`, and `Replace`.
- **Expected Result:** At most 1 pod executes at any single time.
- **Verification:**

```
kubectl get cronjob log-cleaner -n cron-jobs
kubectl get pods -n cron-jobs
```

- **Full Solution:**

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: log-cleaner
  namespace: cron-jobs
spec:
  schedule: "* * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 2
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: cleaner
            image: busybox:1.36
            command: ["/bin/sh", "-c", "echo Cleaning logs; sleep 70"]
```

```
kubectl create namespace cron-jobs
kubectl apply -f cronjob.yaml
```

- **Common Mistakes:** Using deprecated `batch/v1beta1` API group.
- **Troubleshooting:** Inspect events with `kubectl describe cronjob log-cleaner -n cron-jobs`.
- **Cleanup Instructions:**

```
kubectl delete namespace cron-jobs
```

LAB 21: Native Sidecar Containers with `restartPolicy: Always`

- **Difficulty:** Advanced
- **Estimated Duration:** 12 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Implement Kubernetes v1.29+ native sidecar containers in `initContainers` with `restartPolicy: Always`.
- **Prerequisites:** Kubernetes v1.29+ cluster.
- **Cluster Environment:** Modern cluster.
- **Initial State:** Namespace `sidecar-demo`.
- **Task Description:**
  1. Create namespace `sidecar-demo`.
  2. Author a Pod `secure-app` with:
     - An initContainer `auth-proxy` running `busybox:1.36` with `command: ["sleep", "3600"]` and `restartPolicy: Always`.
     - A main application container `web` running `nginx:1.27`.

3. Verify that `auth-proxy` starts first, remains running, and does not block `web` from starting.
4. Verify that `kubectl get pod secure-app` shows `2/2` containers running.

- **Constraints:** Must use native sidecar syntax in `initContainers`.
- **Hints:** Native sidecars are defined inside `spec.initContainers` with `restartPolicy: Always`.
- **Expected Result:** Pod shows `READY: 2/2` and `STATUS: Running`.
- **Verification:**

```
kubectl get pod secure-app -n sidecar-demo
# Expected: READY: 2/2
```

- **Full Solution:**

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
  namespace: sidecar-demo
spec:
  initContainers:
  - name: auth-proxy
    image: busybox:1.36
    command: ["sleep", "3600"]
    restartPolicy: Always
  containers:
  - name: web
    image: nginx:1.27
```

```
kubectl create namespace sidecar-demo
kubectl apply -f sidecar-pod.yaml
```

- **Common Mistakes:** Placing `restartPolicy: Always` on a standard initContainer in older clusters (<v1.28) without the SidecarContainers feature gate.
- **Troubleshooting:** Run `kubectl describe pod secure-app -n sidecar-demo` to inspect initialization sequence.
- **Cleanup Instructions:**

```
kubectl delete namespace sidecar-demo
```

LAB 22: ConfigMap Volume Mounts with Automatic SubPath Updates

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Workloads & Scheduling (15%)

- **Learning Objective:** Mount configuration files from a ConfigMap into a running container without overwriting other files in the destination directory.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `config-lab`.
- **Task Description:**
    1. Create namespace `config-lab`.
    2. Create a ConfigMap `custom-nginx-conf` containing key `custom.conf` with text `server_tokens off;`.
    3. Deploy Pod `nginx-secured` running `nginx:1.27`.
    4. Mount `custom.conf` from the ConfigMap into `/etc/nginx/conf.d/custom.conf` using `subPath` so default nginx config files in that directory are preserved.
    5. Verify the file contents inside the container using `kubectl exec`.
- **Constraints:** Do not overwrite `/etc/nginx/conf.d/default.conf`.
- **Hints:** Use `volumeMounts[*].subPath: custom.conf`.
- **Expected Result:** `/etc/nginx/conf.d/` contains both `default.conf` and `custom.conf`.
- **Verification:**

```
kubectl exec -it nginx-secured -n config-lab -- ls -la /etc/nginx/conf.d/
kubectl exec -it nginx-secured -n config-lab -- cat /etc/nginx/conf.d/custom.conf
```

- **Full Solution:**

```
kubectl create namespace config-lab
kubectl create configmap custom-nginx-conf --from-literal=custom.conf="server_tokens off;" -n confi
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-secured
  namespace: config-lab
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    volumeMounts:
    - name: conf-vol
      mountPath: /etc/nginx/conf.d/custom.conf
      subPath: custom.conf
  volumes:
  - name: conf-vol
    configMap:
      name: custom-nginx-conf
```

```
kubectl apply -f nginx-conf-pod.yaml
```

- **Common Mistakes:** Mounting the volume directly at `/etc/nginx/conf.d/` without `subPath`, which wipes out all existing files in that directory.
- **Troubleshooting:** If pod fails to start with `RunContainerError`, check if the ConfigMap key matches `subPath`.
- **Cleanup Instructions:**

```
kubectl delete namespace config-lab
```

LAB 23: Secret Management and Secure Environment Injection

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Create a generic Kubernetes Secret and inject its keys into an application container as environment variables.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `security-lab`.
- **Task Description:**
    1. Create namespace `security-lab`.
    2. Create a Secret named `database-creds` containing:
        - `DB_USER=app_user`
        - `DB_PASS=SuperSecurePass123!`
    3. Deploy Pod `db-client` running `busybox:1.36` sleeping 3600.
    4. Inject all key-value pairs from `database-creds` as environment variables using `envFrom`.
    5. Verify the environment variables exist inside the container.
- **Constraints:** Do not hardcode secret values into the Pod manifest.
- **Hints:** Use `spec.containers[*].envFrom[*].secretRef`.
- **Expected Result:** `env` inside the pod displays `DB_USER` and `DB_PASS`.
- **Verification:**

```
kubectl exec db-client -n security-lab -- env | grep DB_
```

- **Full Solution:**

```
kubectl create namespace security-lab
kubectl create secret generic database-creds \
  --from-literal=DB_USER=app_user \
  --from-literal=DB_PASS='SuperSecurePass123!' \
  -n security-lab
```

```
apiVersion: v1
kind: Pod
metadata:
  name: db-client
  namespace: security-lab
spec:
  containers:
  - name: client
    image: busybox:1.36
    command: ["sleep", "3600"]
    envFrom:
    - secretRef:
        name: database-creds
```

```
kubectl apply -f db-client-pod.yaml
```

- **Common Mistakes:** Using `env` with single keys when `envFrom` can import all keys in one declaration.
- **Troubleshooting:** Run `kubectl describe pod db-client -n security-lab` if pod fails to launch.
- **Cleanup Instructions:**

```
kubectl delete namespace security-lab
```

LAB 24: Pod Quality of Service (QoS) Classes

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Author three separate Pod specifications to explicitly produce the three Kubernetes QoS classes: Guaranteed, Burstable, and BestEffort.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `qos-lab`.
- **Task Description:**
  1. Create namespace `qos-lab`.
  2. Create Pod `pod-guaranteed`: Requests and Limits must be identical for both CPU and Memory (e.g. 100m / 128Mi).
  3. Create Pod `pod-burstable`: Request 100m CPU / 128Mi RAM, Limit 200m CPU / 256Mi RAM.
  4. Create Pod `pod-besteffort`: No requests and no limits defined.
  5. Inspect `status.qosClass` on all three pods.
- **Constraints:** Do not use default namespace LimitRanges that might inject requests.
- **Hints:** Guaranteed requires both CPU and RAM requests to equal limits on every container.
- **Expected Result:** `kubectl get pods` shows `Guaranteed`, `Burstable`, and `BestEffort` respectively.

- **Verification:**

```
kubectl get pods -n qos-lab -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass
```

- **Full Solution:**

```
kubectl create namespace qos-lab
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-guaranteed
  namespace: qos-lab
spec:
  containers:
  - name: c1
    image: busybox:1.36
    command: ["sleep", "3600"]
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 100m
        memory: 128Mi
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-burstable
  namespace: qos-lab
spec:
  containers:
  - name: c1
    image: busybox:1.36
    command: ["sleep", "3600"]
    resources:
      requests:
        cpu: 100m
        memory: 128Mi
      limits:
        cpu: 200m
        memory: 256Mi
---
apiVersion: v1
kind: Pod
metadata:
  name: pod-besteffort
  namespace: qos-lab
spec:
  containers:
  - name: c1
    image: busybox:1.36
    command: ["sleep", "3600"]
```

```
kubectl apply -f qos-pods.yaml
```

- **Common Mistakes:** Defining requests and limits for CPU but omitting memory limits in the Guaranteed pod.
- **Troubleshooting:** If `status.qosClass` is Burstable when aiming for Guaranteed, verify memory limits are explicitly set.
- **Cleanup Instructions:**

```
kubectl delete namespace qos-lab
```

LAB 25: Multi-Tier Health Checking (Startup, Readiness, Liveness)

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Configure a resilient Pod with startup, readiness, and liveness probes to prevent premature restarts and bad traffic routing.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `probe-lab`.
- **Task Description:**
  1. Create namespace `probe-lab`.
  2. Deploy Pod `robust-web` running image `nginx:1.27`.
  3. Add a `startupProbe` checking HTTP path `/` on port 80 with `failureThreshold: 30` and `periodSeconds: 2`.
  4. Add a `readinessProbe` checking HTTP path `/` on port 80 with `initialDelaySeconds: 5` and `periodSeconds: 5`.
  5. Add a `livenessProbe` checking HTTP path `/` on port 80 with `initialDelaySeconds: 10` and `periodSeconds: 15`.
  6. Expose the Pod with a ClusterIP Service and verify the endpoint is populated only after readiness passes.
- **Constraints:** Do not use probe paths that return HTTP 404 or 500.
- **Hints:** Startup probe protects slow apps; liveness probe detects deadlock.
- **Expected Result:** Pod reaches `1/1 Running`, readiness probe succeeds, endpoint exists.
- **Verification:**

```
kubectl describe pod robust-web -n probe-lab | grep -i probe
kubectl get endpoints robust-web-svc -n probe-lab
```

- **Full Solution:**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: robust-web
  namespace: probe-lab
  labels:
    app: robust-web
spec:
  containers:
  - name: nginx
    image: nginx:1.27
    ports:
    - containerPort: 80
    startupProbe:
      httpGet:
        path: /
        port: 80
      failureThreshold: 30
      periodSeconds: 2
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 5
    livenessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: robust-web-svc
  namespace: probe-lab
spec:
  selector:
    app: robust-web
  ports:
  - port: 80
    targetPort: 80
```

```
kubectl create namespace probe-lab
kubectl apply -f robust-web.yaml
```

- **Common Mistakes:** Setting port number incorrectly or testing a non-existent path like `/healthz` on vanilla NGINX.
- **Troubleshooting:** Run `kubectl describe pod robust-web -n probe-lab` and look for `Probe Failed` events.
- **Cleanup Instructions:**

```
kubectl delete namespace probe-lab
```

LAB 26: Node Affinity with Required and Preferred Matching

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Author advanced Node Affinity rules combining hard (`required`) requirements with soft (`preferred`) weighted scoring.
- **Prerequisites:** Multi-node cluster.
- **Cluster Environment:** Multi-worker cluster.
- **Initial State:** Label `worker-1` with `disktype=nvme` and `region=us-central` . Label `worker-2` with `disktype=hdd` and `region=us-central` .
- **Task Description:**
    1. Label worker nodes as described.
    2. Deploy a Pod `analytics-engine` running `alpine` sleeping 3600.
    3. Configure Node Affinity:
        - `requiredDuringSchedulingIgnoredDuringExecution` : Must be in `region=us-central` .
        - `preferredDuringSchedulingIgnoredDuringExecution` : Prefer `disktype=nvme` with weight 100.
    4. Verify the Pod schedules onto `worker-1` due to the preferred score.
- **Constraints:** If `worker-1` is deleted or cordoned, the Pod must still be capable of landing on `worker-2` .
- **Hints:** Use `nodeSelectorTerms` for required and `preferences` for preferred.
- **Expected Result:** Pod lands on `worker-1` .
- **Verification:**

```
kubectl get pod analytics-engine -o wide
# Node should be worker-1
```

- **Full Solution:**

```
kubectl label nodes worker-1 disktype=nvme region=us-central --overwrite
kubectl label nodes worker-2 disktype=hdd region=us-central --overwrite
```

```
apiVersion: v1
kind: Pod
metadata:
  name: analytics-engine
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: region
            operator: In
            values:
            - us-central
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
          - key: disktype
            operator: In
            values:
            - nvme
  containers:
  - name: engine
    image: alpine
    command: ["sleep", "3600"]
```

```
kubectl apply -f affinity-pod.yaml
```

- **Common Mistakes:** Misspelling `requiredDuringSchedulingIgnoredDuringExecution`.
- **Troubleshooting:** Run `kubectl describe pod analytics-engine` if scheduling fails.
- **Cleanup Instructions:**

```
kubectl delete pod analytics-engine
kubectl label nodes worker-1 disktype- region-
kubectl label nodes worker-2 disktype- region-
```

LAB 27: Taints, Tolerations, and Node Eviction (`NoExecute`)

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Observe the eviction behavior of `NoExecute` taints versus `NoSchedule`, and use `tolerationSeconds` for graceful drain windows.
- **Prerequisites:** Multi-node cluster.
- **Cluster Environment:** Multi-worker cluster.
- **Initial State:** Running Deployment `stateless-app` with 3 replicas on `worker-1`.

- **Task Description:**
    1. Taint `worker-1` with `outage=maintenance:NoExecute`.
    2. Observe that all non-tolerating pods on `worker-1` are immediately evicted and recreated on other nodes.
    3. Deploy a Pod `tolerant-pod` that tolerates `outage=maintenance:NoExecute` with `tolerationSeconds: 120`.
    4. Verify `tolerant-pod` runs on `worker-1` and remains alive for 2 minutes before eviction.
- **Constraints:** Understand the difference between `NoSchedule` (prevents new pods) and `NoExecute` (kills running pods).
- **Hints:** Trailing minus sign removes a taint: `kubectl taint nodes <node> <key>:<effect>-`.
- **Expected Result:** `NoExecute` clears un-tolerating pods from the node.
- **Verification:**

```
kubectl get pods -o wide | grep worker-1
```

- **Full Solution:**

```
kubectl taint nodes worker-1 outage=maintenance:NoExecute
```

```
apiVersion: v1
kind: Pod
metadata:
  name: tolerant-pod
spec:
  nodeName: worker-1
  tolerations:
  - key: "outage"
    operator: "Equal"
    value: "maintenance"
    effect: "NoExecute"
    tolerationSeconds: 120
  containers:
  - name: app
    image: alpine
    command: ["sleep", "3600"]
```

```
kubectl apply -f tolerant-pod.yaml
# Remove taint after verification
kubectl taint nodes worker-1 outage=maintenance:NoExecute-
```

- **Common Mistakes:** Applying `NoSchedule` and expecting running pods to be evicted (only `NoExecute` evicts).
- **Troubleshooting:** Describe the node to verify active taints: `kubectl describe node worker-1 | grep Taints`.
- **Cleanup Instructions:**

```
kubectl delete pod tolerant-pod
kubectl taint nodes worker-1 outage- || true
```

LAB 28: Pod Anti-Affinity Across Topology Failure Domains

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Guarantee high availability by authoring strict `podAntiAffinity` rules spreading replicas across distinct physical host nodes.
- **Prerequisites:** Multi-node cluster (at least 2 worker nodes).
- **Cluster Environment:** Multi-worker cluster.
- **Initial State:** Namespace `ha-demo`.
- **Task Description:**
    1. Create namespace `ha-demo`.
    2. Deploy a Deployment `redundant-api` with 2 replicas in `ha-demo`.
    3. Image: `nginx:1.27`.
    4. Configure `podAntiAffinity` with `requiredDuringSchedulingIgnoredDuringExecution` matching `app: redundant-api` on `topologyKey: kubernetes.io/hostname`.
    5. Verify that each replica is scheduled on a completely different worker node.
    6. Scale the Deployment to 4 replicas (more than available worker nodes) and observe that surplus pods remain `Pending`.
- **Constraints:** Must use hard `requiredDuringSchedulingIgnoredDuringExecution`.
- **Hints:** The topologyKey `kubernetes.io/hostname` is standard for per-node spreading.
- **Expected Result:** Exactly 1 pod per node; surplus pods show `Pending`.
- **Verification:**

```
kubectl get pods -n ha-demo -o wide
```

- **Full Solution:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redundant-api
  namespace: ha-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: redundant-api
  template:
    metadata:
      labels:
        app: redundant-api
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
          - labelSelector:
              matchExpressions:
              - key: app
                operator: In
                values:
                - redundant-api
            topologyKey: "kubernetes.io/hostname"
      containers:
      - name: nginx
        image: nginx:1.27
```

```
kubectl create namespace ha-demo
kubectl apply -f anti-affinity.yaml
kubectl scale deployment redundant-api --replicas=4 -n ha-demo
```

- **Common Mistakes:** Omitting the `matchLabels` in the `podAntiAffinity.labelSelector`.
- **Troubleshooting:** Run `kubectl describe pod <pending-pod> -n ha-demo` to see `FailedScheduling: 0/2 nodes are available: 2 node(s) had untolerated taint or pod anti-affinity rules`.
- **Cleanup Instructions:**

```
kubectl delete namespace ha-demo
```

LAB 29: Horizontal Pod Autoscaler (HPA) Tuning with Metrics Server

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Workloads & Scheduling (15%)

- **Learning Objective:** Deploy and configure an HPA resource using `autoscaling/v2` to autoscale a workload between 2 and 8 replicas based on CPU load.
- **Prerequisites:** Metrics Server installed (`kubectl top pods` works).
- **Cluster Environment:** Standard Kubernetes cluster with Metrics Server.
- **Initial State:** Namespace `scaling-lab` .
- **Task Description:**
    1. Create namespace `scaling-lab` .
    2. Deploy `cpu-stress` Deployment with 2 replicas of `nginx:1.27` , setting `resources.requests.cpu: 50m` and `resources.limits.cpu: 100m` .
    3. Expose Deployment with a Service `cpu-stress-svc` .
    4. Create an HPA `cpu-stress-hpa` targeting 60% CPU utilization with minReplicas 2 and maxReplicas 8.
    5. Inspect the HPA status and verify metrics are reading.
- **Constraints:** Deployment must have resource requests declared or HPA will report `<unknown>` .
- **Hints:** Use `kubectl autoscale deployment cpu-stress --cpu-percent=60 --min=2 --max=8 -n scaling-lab` .
- **Expected Result:** HPA shows active target metrics (e.g. `0%/60%`).
- **Verification:**

```
kubectl get hpa cpu-stress-hpa -n scaling-lab
```

- **Full Solution:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cpu-stress
  namespace: scaling-lab
spec:
  replicas: 2
  selector:
    matchLabels:
      app: cpu-stress
  template:
    metadata:
      labels:
        app: cpu-stress
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        resources:
          requests:
            cpu: 50m
          limits:
            cpu: 100m
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: cpu-stress-hpa
  namespace: scaling-lab
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: cpu-stress
  minReplicas: 2
  maxReplicas: 8
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
```

```
kubectl create namespace scaling-lab
kubectl apply -f hpa-lab.yaml
```

- **Common Mistakes:** Forgetting to define `resources.requests.cpu` on the Deployment containers.
- **Troubleshooting:** If HPA reports `<unknown>/60%`, wait 60 seconds for metrics-server scrape interval, or check `kubectl top pods -n scaling-lab`.
- **Cleanup Instructions:**

```
kubectl delete namespace scaling-lab
```

LAB 30: Namespace ResourceQuotas and Container LimitRanges

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Workloads & Scheduling (15%)
- **Learning Objective:** Configure interlocking `ResourceQuota` and `LimitRange` policies to enforce namespace resource caps while preventing missing-request admission rejections.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `dev-sandbox`.
- **Task Description:**
    1. Create namespace `dev-sandbox`.
    2. Create a `ResourceQuota` named `compute-quota` capping total CPU requests at `1`, CPU limits at `2`, and maximum Pods at `4`.
    3. Create a `LimitRange` named `container-defaults` setting default request to `100m` CPU / `128Mi` RAM and default limit to `200m` CPU / `256Mi` RAM.
    4. Launch a Pod without specifying requests/limits (`kubectl run test-pod --image=nginx:alpine -n dev-sandbox`).
    5. Verify the Pod is accepted and has the default limits injected by the LimitRange.
    6. Verify quota usage reflects the new pod.
- **Constraints:** Do not allow unconstrained pods in `dev-sandbox`.
- **Hints:** Without the LimitRange, `kubectl run` fails immediately against a ResourceQuota!
- **Expected Result:** `test-pod` runs, and `kubectl describe quota -n dev-sandbox` shows 1/4 pods used.
- **Verification:**

```
kubectl describe quota compute-quota -n dev-sandbox
kubectl get pod test-pod -n dev-sandbox -o jsonpath='{.spec.containers[0].resources}'
```

- **Full Solution:**

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev-sandbox
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev-sandbox
spec:
  hard:
    requests.cpu: "1"
    limits.cpu: "2"
    pods: "4"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: container-defaults
  namespace: dev-sandbox
spec:
  limits:
  - default:
      cpu: 200m
      memory: 256Mi
    defaultRequest:
      cpu: 100m
      memory: 128Mi
    type: Container
```

```
kubectl apply -f quota-limit.yaml
kubectl run test-pod --image=nginx:alpine -n dev-sandbox
```

- **Common Mistakes:** Applying a ResourceQuota without a LimitRange, causing subsequent developer deployments without explicit resource blocks to fail admission.
- **Troubleshooting:** Run `kubectl describe quota compute-quota -n dev-sandbox` to see available vs used headroom.
- **Cleanup Instructions:**

```
kubectl delete namespace dev-sandbox
```

VOLUME II: THE CKA PRACTICAL WORKBOOK

DOMAIN 3: SERVICES & NETWORKING (LABS 31–45)

LAB 31: ClusterIP Service Creation and Endpoint Verification

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Create a ClusterIP Service to expose a backend Deployment and verify automatic endpoint generation.
- **Prerequisites:** Running cluster.
- **Cluster Environment:** Standard Kubernetes cluster.
- **Initial State:** Clean namespace `networking-lab`.
- **Task Description:**
    1. Create namespace `networking-lab`.
    2. Deploy a Deployment `backend-api` with 3 replicas of `nginx:1.27` in `networking-lab`.
    3. Create a ClusterIP Service named `backend-svc` exposing port 80 targeting containerPort 80.
    4. Inspect the Service endpoints and confirm all 3 Pod IPs appear in the endpoints list.
    5. Test internal connectivity to `backend-svc.networking-lab.svc.cluster.local` using a curl test pod.
- **Constraints:** Service must be of type `ClusterIP`.
- **Hints:** Use `kubectl expose deployment backend-api --name=backend-svc --port=80 --target-port=80 -n networking-lab`.
- **Expected Result:** `backend-svc` resolves internally and load balances across all 3 backend pods.
- **Verification:**

```
kubectl get endpoints backend-svc -n networking-lab
kubectl run curl-test --image=curlimages/curl --rm -it --restart=Never -n networking-lab -- http://
```

- **Full Solution:**

```
kubectl create namespace networking-lab
kubectl create deployment backend-api --image=nginx:1.27 --replicas=3 -n networking-lab
kubectl expose deployment backend-api --name=backend-svc --port=80 --target-port=80 -n networking-l
```

- **Common Mistakes:** Typo in the selector labels causing empty endpoints (`<none>`).
- **Troubleshooting:** Compare `kubectl get svc backend-svc -n networking-lab -o yaml` with `kubectl get pods -n networking-lab --show-labels` .
- **Cleanup Instructions:**

```
kubectl delete namespace networking-lab
```

LAB 32: NodePort Service Configuration on Static Ports

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Expose an application externally on a specific host NodePort within the range 30000–32767.
- **Prerequisites:** Multi-node or local cluster with port forwarding.
- **Cluster Environment:** Any cluster.
- **Initial State:** Deployment `web-portal` exists in `default` .
- **Task Description:**
    1. Create Deployment `web-portal` with 2 replicas of `nginx:1.27` .
    2. Create a NodePort Service named `web-portal-external` with:
        - `port: 80`
        - `targetPort: 80`
        - `nodePort: 32080`
    3. Query the NodePort on the worker node's IP using `curl` to verify accessibility.
- **Constraints:** `nodePort` must be explicitly set to 32080.
- **Hints:** NodePort range is 30000–32767.
- **Expected Result:** HTTP 200 OK received when curling `http://<node-ip>:32080` .
- **Verification:**

```
NODE_IP=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[?(@.type=="InternalIP")].addre
curl -I http://${NODE_IP}:32080
```

- **Full Solution:**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-portal
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-portal
  template:
    metadata:
      labels:
        app: web-portal
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
---
apiVersion: v1
kind: Service
metadata:
  name: web-portal-external
spec:
  type: NodePort
  selector:
    app: web-portal
  ports:
  - port: 80
    targetPort: 80
    nodePort: 32080
```

```
kubectl apply -f nodeport-lab.yaml
```

- **Common Mistakes:** Specifying a `nodePort` below 30000 or above 32767, which fails API validation.
- **Troubleshooting:** Check if host firewall (ufw or iptables) is blocking traffic on port 32080.
- **Cleanup Instructions:**

```
kubectl delete -f nodeport-lab.yaml
```

LAB 33: Headless Service for Stateful Database Peer Discovery

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Create a Headless Service (`clusterIP: None`) and observe DNS A-record generation for individual StatefulSet Pods.
- **Prerequisites:** StatefulSet running.
- **Cluster Environment:** Any cluster.

- **Initial State:** Namespace `cluster-db`.
- **Task Description:**
    1. Create namespace `cluster-db`.
    2. Create a headless Service `db-discovery` with `clusterIP: None` in `cluster-db` targeting `role: datastore`.
    3. Deploy a StatefulSet `datastore` with 2 replicas using `redis:7.2-alpine` attached to `serviceName: db-discovery`.
    4. Launch an interactive test pod with `nslookup` to verify that `db-discovery.cluster-db.svc.cluster.local` returns direct IPs for both `datastore-0` and `datastore-1`.
    5. Verify direct lookup for `datastore-0.db-discovery.cluster-db.svc.cluster.local`.
- **Constraints:** `spec.clusterIP` must be set to `None`.
- **Hints:** Headless services bypass kube-proxy load balancing entirely.
- **Expected Result:** DNS returns individual pod IP records.
- **Verification:**

```
kubectl run dns-test --image=busybox:1.36 --rm -it --restart=Never -n cluster-db -- nslookup db-dis
```

- **Full Solution:**

```
apiVersion: v1
kind: Namespace
metadata:
  name: cluster-db
---
apiVersion: v1
kind: Service
metadata:
  name: db-discovery
  namespace: cluster-db
spec:
  clusterIP: None
  selector:
    role: datastore
  ports:
  - port: 6379
    name: redis
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: datastore
  namespace: cluster-db
spec:
  serviceName: db-discovery
  replicas: 2
  selector:
    matchLabels:
      role: datastore
  template:
    metadata:
      labels:
        role: datastore
    spec:
      containers:
      - name: redis
        image: redis:7.2-alpine
```

```
kubectl apply -f headless-db.yaml
```

- **Common Mistakes:** Missing `serviceName` in StatefulSet spec, which breaks individual pod DNS records.
- **Troubleshooting:** Run `kubectl get endpoints db-discovery -n cluster-db` to verify registered endpoints.
- **Cleanup Instructions:**

```
kubectl delete namespace cluster-db
```

LAB 34: EndpointSlice Inspection and Manual Endpoint Injection

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Create a Service without selectors and manually attach an external IP address via `Endpoints` and `EndpointSlices`.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** External legacy database at `192.168.1.50:3306`.
- **Task Description:**
    1. Create a Service named `external-mysql` with no `spec.selector` on port 3306 targeting port 3306.
    2. Create an `Endpoints` object with the exact same name `external-mysql` pointing to IP `192.168.1.50:3306`.
    3. Verify that Kubernetes automatically generates a matching `EndpointSlice` for the Service.
- **Constraints:** Do not use `spec.selector` on the Service.
- **Hints:** When a Service omits `selector`, Kubernetes does not create endpoints automatically, allowing manual declaration.
- **Expected Result:** `kubectl describe svc external-mysql` lists `Endpoints: 192.168.1.50:3306`.
- **Verification:**

```
kubectl get endpoints external-mysql
kubectl get endpointslices -l kubernetes.io/service-name=external-mysql
```

- **Full Solution:**

```
apiVersion: v1
kind: Service
metadata:
  name: external-mysql
spec:
  ports:
  - port: 3306
    targetPort: 3306
---
apiVersion: v1
kind: Endpoints
metadata:
  name: external-mysql
subsets:
- addresses:
  - ip: 192.168.1.50
  ports:
  - port: 3306
```

```
kubectl apply -f manual-endpoints.yaml
```

- **Common Mistakes:** Naming the `Endpoints` object differently than the `Service` object. They must match exactly!
- **Troubleshooting:** Run `kubectl describe endpoints external-mysql` to check address format.
- **Cleanup Instructions:**

```
kubectl delete svc external-mysql
kubectl delete endpoints external-mysql
```

LAB 35: CoreDNS Resolution Triage and Custom Upstream Forwarding

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Modify the CoreDNS ConfigMap `Corefile` to add custom upstream DNS forwarders and verify cache behavior.
- **Prerequisites:** Cluster admin access.
- **Cluster Environment:** Any cluster.
- **Initial State:** Default CoreDNS ConfigMap in `kube-system`.
- **Task Description:**
  1. Backup the existing `coredns` ConfigMap in `kube-system` to `/tmp/coredns-backup.yaml`.
  2. Edit `coredns` ConfigMap to configure upstream DNS forwarder `forward . 8.8.8.8 1.1.1.1`.
  3. Restart CoreDNS pods to pick up the updated Corefile.
  4. Launch a test pod and verify resolution of an external domain (e.g., `google.com`) and internal Service (`kubernetes.default`).
- **Constraints:** Do not corrupt the Corefile syntax.
- **Hints:** CoreDNS automatically reloads within 30 seconds, or restart pods with `kubectl rollout restart deployment coredns -n kube-system`.
- **Expected Result:** DNS resolution succeeds for both internal and external queries.
- **Verification:**

```
kubectl run dns-check --image=busybox:1.36 --rm -it --restart=Never -- nslookup google.com
kubectl run dns-check --image=busybox:1.36 --rm -it --restart=Never -- nslookup kubernetes.default
```

- **Full Solution:**

```
kubectl get configmap coredns -n kube-system -o yaml > /tmp/coredns-backup.yaml
# Modify forward line in Corefile to: forward . 8.8.8.8 1.1.1.1
kubectl edit configmap coredns -n kube-system
kubectl rollout restart deployment coredns -n kube-system
kubectl rollout status deployment coredns -n kube-system
```

- **Common Mistakes:** Introducing syntax errors in Corefile curly braces `{}` causing CoreDNS crash loop.
- **Troubleshooting:** Check logs: `kubectl logs -n kube-system -l k8s-app=kube-dns`.
- **Cleanup Instructions:**

```
kubectl apply -f /tmp/coredns-backup.yaml
kubectl rollout restart deployment coredns -n kube-system
```

LAB 36: Ingress Controller Routing with Path-Based Dispatching

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Author an Ingress resource with `pathType: Prefix` dispatching traffic across two independent microservices.
- **Prerequisites:** Ingress Controller running.
- **Cluster Environment:** Standard cluster with ingress-nginx.
- **Initial State:** Namespace `ecommerce`.
- **Task Description:**
  1. Create namespace `ecommerce`.
  2. Deploy `catalog-api` (1 replica, image `nginx:1.27`, port 80) and expose with Service `catalog-svc`.
  3. Deploy `orders-api` (1 replica, image `httpd:alpine`, port 80) and expose with Service `orders-svc`.
  4. Create an Ingress `shop-ingress` in `ecommerce` using `ingressClassName: nginx` matching host `shop.example.local`:
     - Path `/catalog` $\to$ `catalog-svc:80`
     - Path `/orders` $\to$ `orders-svc:80`
  5. Test URL dispatching with `curl` using the Host header.
- **Constraints:** Use `pathType: Prefix`.
- **Hints:** Pass `--header "Host: shop.example.local"` to `curl`.
- **Expected Result:** Traffic to `/catalog` reaches `catalog-api`; traffic to `/orders` reaches `orders-api`.
- **Verification:**

```
kubectl get ingress shop-ingress -n ecommerce
```

- **Full Solution:**

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop-ingress
  namespace: ecommerce
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: shop.example.local
    http:
      paths:
      - path: /catalog
        pathType: Prefix
        backend:
          service:
            name: catalog-svc
            port:
              number: 80
      - path: /orders
        pathType: Prefix
        backend:
          service:
            name: orders-svc
            port:
              number: 80
```

```
kubectl create namespace ecommerce
kubectl create deployment catalog-api --image=nginx:1.27 -n ecommerce
kubectl expose deployment catalog-api --name=catalog-svc --port=80 -n ecommerce
kubectl create deployment orders-api --image=httpd:alpine -n ecommerce
kubectl expose deployment orders-api --name=orders-svc --port=80 -n ecommerce
kubectl apply -f ingress-lab.yaml
```

- **Common Mistakes:** Omitting the rewrite-target annotation when backend containers expect `/` root paths.
- **Troubleshooting:** Run `kubectl describe ingress shop-ingress -n ecommerce` to verify backend endpoints.
- **Cleanup Instructions:**

```
kubectl delete namespace ecommerce
```

LAB 37: Ingress TLS Termination with Cryptographic Secrets

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Generate a self-signed TLS certificate pair, create a `kubernetes.io/tls` Secret, and configure Ingress SSL termination.
- **Prerequisites:** OpenSSL installed on workstation.
- **Cluster Environment:** Ingress-enabled cluster.
- **Initial State:** Service `secure-portal-svc` running in `default`.
- **Task Description:**
    1. Generate a self-signed TLS cert and private key for domain `portal.company.com`:
        - `tls.crt` and `tls.key`.
    2. Create a TLS Secret named `portal-tls-secret` from the cert and key.
    3. Create an Ingress resource `secure-portal-ingress` terminating TLS for `portal.company.com` using `portal-tls-secret`.
    4. Verify that HTTPS requests to the Ingress controller succeed.
- **Constraints:** Secret must be of type `kubernetes.io/tls`.
- **Hints:** Use `kubectl create secret tls portal-tls-secret --cert=tls.crt --key=tls.key`.
- **Expected Result:** Ingress terminates TLS cleanly on port 443.
- **Verification:**

```
kubectl get secret portal-tls-secret -o yaml
kubectl describe ingress secure-portal-ingress | grep -A 5 TLS
```

- **Full Solution:**

```
# 1. Generate cert
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /tmp/tls.key -out /tmp/tls.crt \
  -subj "/CN=portal.company.com/O=Company"

# 2. Create Secret
kubectl create secret tls portal-tls-secret --cert=/tmp/tls.crt --key=/tmp/tls.key

# 3. Create Ingress
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: secure-portal-ingress
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - portal.company.com
    secretName: portal-tls-secret
  rules:
  - host: portal.company.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-portal-svc
            port:
              number: 80
EOF
```

- **Common Mistakes:** Creating a generic secret instead of a TLS secret (`kubectl create secret generic` lacks required data fields `tls.crt` and `tls.key`).
- **Troubleshooting:** Verify Secret keys with `kubectl describe secret portal-tls-secret` .
- **Cleanup Instructions:**

```
kubectl delete ingress secure-portal-ingress
kubectl delete secret portal-tls-secret
rm -f /tmp/tls.key /tmp/tls.crt
```

LAB 38: Next-Gen Traffic Management with Gateway API (`GatewayClass` & `Gateway`)

- **Difficulty:** Advanced
- **Estimated Duration:** 15 minutes
- **Domain:** Services & Networking (20%)

- **Learning Objective:** Inspect available GatewayClasses and deploy a modern Gateway resource declaring HTTP and HTTPS entry point listeners.
- **Prerequisites:** Gateway API CRDs installed on cluster.
- **Cluster Environment:** Standard Kubernetes v1.31+ cluster.
- **Initial State:** GatewayClass `cilium` or `envoy` installed.
- **Task Description:**
    1. Verify installed GatewayClasses using `kubectl get gatewayclasses`.
    2. Create namespace `gateway-infra`.
    3. Author a `Gateway` resource named `edge-gateway` in namespace `gateway-infra`.
    4. Configure two listeners:
        - `http-listener`: port 80, protocol HTTP, allowedRoutes from all namespaces.
        - `https-listener`: port 443, protocol HTTPS, allowedRoutes from same namespace.
    5. Inspect the Gateway status conditions to verify it transitions to `Programmed: True`.
- **Constraints:** Must use `gateway.networking.k8s.io/v1`.
- **Hints:** Check `kubectl explain gateway.spec.listeners`.
- **Expected Result:** `edge-gateway` is created and reports Programmed condition.
- **Verification:**

```
kubectl get gateway edge-gateway -n gateway-infra
```

- **Full Solution:**

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: edge-gateway
  namespace: gateway-infra
spec:
  gatewayClassName: eg-gateway-class
  listeners:
  - name: http-listener
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: All
  - name: https-listener
    port: 443
    protocol: HTTPS
    allowedRoutes:
      namespaces:
        from: Same
    tls:
      mode: Terminate
      certificateRefs:
      - name: edge-tls-cert
```

```
kubectl create namespace gateway-infra
kubectl apply -f gateway.yaml
```

- **Common Mistakes:** Referencing a non-existent `gatewayClassName`.
- **Troubleshooting:** Run `kubectl describe gateway edge-gateway -n gateway-infra` to check listener validation errors.
- **Cleanup Instructions:**

```
kubectl delete namespace gateway-infra
```

LAB 39: Gateway API `HTTPRoute` with Weighted Traffic Splitting

- **Difficulty:** Advanced
- **Estimated Duration:** 15 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Implement a canary release using Gateway API `HTTPRoute` with a 90/10 weighted backend traffic split.
- **Prerequisites:** Gateway resource `prod-gateway` running.
- **Cluster Environment:** Gateway API enabled cluster.
- **Initial State:** Namespace `canary-demo` with Services `web-v1-svc` and `web-v2-svc`.
- **Task Description:**
  1. In namespace `canary-demo`, create two Deployments: `web-v1` (image `nginx:1.26`) and `web-v2` (image `nginx:1.27`).
  2. Expose both with ClusterIP Services `web-v1-svc` and `web-v2-svc` on port 80.
  3. Create an `HTTPRoute` named `web-canary-route` attaching to parent Gateway `prod-gateway`.
  4. Configure path `/` matching hostname `app.company.internal`.
  5. Split traffic: 90% weight to `web-v1-svc`, 10% weight to `web-v2-svc`.
  6. Verify route status condition shows `Accepted: True`.
- **Constraints:** Do not use legacy Ingress annotations.
- **Hints:** Use `spec.rules[*].backendRefs[*].weight`.
- **Expected Result:** HTTPRoute distributes traffic across backends proportionally to weights.
- **Verification:**

```
kubectl describe httproute web-canary-route -n canary-demo | grep -A 8 Rules
```

- **Full Solution:**

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: web-canary-route
  namespace: canary-demo
spec:
  parentRefs:
  - name: prod-gateway
  hostnames:
  - "app.company.internal"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: web-v1-svc
      port: 80
      weight: 90
    - name: web-v2-svc
      port: 80
      weight: 10
```

```
kubectl apply -f canary-route.yaml
```

- **Common Mistakes:** Summing weights to something other than 100 or using negative integers.
- **Troubleshooting:** Run `kubectl get httproute web-canary-route -n canary-demo -o yaml` and check `status.parents`.
- **Cleanup Instructions:**

```
kubectl delete namespace canary-demo
```

LAB 40: Gateway API Cross-Namespace Route Attachment

- **Difficulty:** Advanced
- **Estimated Duration:** 12 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Securely attach an `HTTPRoute` residing in namespace `apps` to a centralized `Gateway` in namespace `infra` using `allowedRoutes`.
- **Prerequisites:** Multi-namespace Gateway API setup.
- **Cluster Environment:** Gateway API enabled cluster.
- **Initial State:** Namespaces `infra` and `apps` exist.
- **Task Description:**
  1. In namespace `infra`, create Gateway `shared-gw` whose HTTP listener explicitly allows routes from namespaces with label `tenant: allowed`.
  2. Label namespace `apps` with `tenant: allowed`.

3. In namespace `apps`, create `HTTPRoute` named `tenant-route` referencing parent Gateway `shared-gw` in namespace `infra`.
4. Verify the route is accepted by the gateway.

- **Constraints:** Route attachment must be rejected if the namespace label is removed.
- **Hints:** Use `allowedRoutes.namespaces.from: Selector` on the Gateway listener.
- **Expected Result:** `tenant-route` shows `Accepted: True` in conditions.
- **Verification:**

```
kubectl describe httproute tenant-route -n apps | grep "Accepted:"
```

- **Full Solution:**

```
kubectl create namespace infra
kubectl create namespace apps
kubectl label namespace apps tenant=allowed
```

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: shared-gw
  namespace: infra
spec:
  gatewayClassName: eg-gateway-class
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Selector
        selector:
          matchLabels:
            tenant: allowed
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: tenant-route
  namespace: apps
spec:
  parentRefs:
  - name: shared-gw
    namespace: infra
  rules:
  - backendRefs:
    - name: app-svc
      port: 80
```

- **Common Mistakes:** Forgetting `parentRefs[*].namespace: infra` when attaching from a foreign namespace.

- **Troubleshooting:** If status is `Refused`, check namespace label with `kubectl get ns apps --show-labels`.
- **Cleanup Instructions:**

```
kubectl delete namespace infra apps
```

LAB 41: Default-Deny All Ingress NetworkPolicy Enforcement

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Lock down an entire namespace by applying a default-deny Ingress NetworkPolicy, and observe immediate traffic drop.
- **Prerequisites:** Cluster with policy-enforcing CNI (Calico or Cilium).
- **Cluster Environment:** NetworkPolicy capable cluster.
- **Initial State:** Namespace `isolated-ns` with a working NGINX pod.
- **Task Description:**
  1. Create namespace `isolated-ns`.
  2. Deploy Pod `web` (`nginx:1.27`) and expose with Service `web-svc` in `isolated-ns`.
  3. Verify a curl pod in `default` can reach `web-svc.isolated-ns`.
  4. Author and apply a NetworkPolicy `default-deny-ingress` in `isolated-ns` selecting all pods (`podSelector: {}`) with `policyTypes: [Ingress]`.
  5. Verify that subsequent curls from `default` immediately hang and time out.
- **Constraints:** Do not block egress traffic.
- **Hints:** `podSelector: {}` acts as a wildcard matching all pods in that namespace.
- **Expected Result:** Incoming connections are dropped at the CNI layer.
- **Verification:**

```
kubectl run test-curl --image=curlimages/curl --rm -it --restart=Never -- --connect-timeout 3 http:
# Expected: Connection timed out
```

- **Full Solution:**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: isolated-ns
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

```
kubectl create namespace isolated-ns
kubectl run web --image=nginx:1.27 -n isolated-ns --port=80
kubectl expose pod web --name=web-svc --port=80 -n isolated-ns
kubectl apply -f default-deny.yaml
```

- **Common Mistakes:** Testing on a cluster running Flannel without Calico, where NetworkPolicies are silently ignored.
- **Troubleshooting:** Run `kubectl describe netpol default-deny-ingress -n isolated-ns` to confirm `Allowing ingress traffic: <none>`.
- **Cleanup Instructions:**

```
kubectl delete namespace isolated-ns
```

LAB 42: Inter-Pod Ingress Filtering with `podSelector` and `namespaceSelector`

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Implement a fine-grained Ingress rule allowing traffic to database pods strictly from frontend pods in a production namespace.
- **Prerequisites:** NetworkPolicy capable CNI.
- **Cluster Environment:** Any test cluster.
- **Initial State:** Namespaces `frontend-ns` and `backend-ns`.
- **Task Description:**
    1. Create namespace `frontend-ns` and label it `env: prod`.
    2. Create namespace `backend-ns`.
    3. In `backend-ns`, run Pod `db` with label `app: db` on port 5432.
    4. Author a NetworkPolicy `allow-frontend-to-db` in `backend-ns`:
        - Targets pods labeled `app: db`.
        - Allows ingress on TCP port 5432 strictly from pods labeled `app: frontend` residing in namespaces labeled `env: prod`.
    5. Verify pods with other labels cannot connect.
- **Constraints:** Enforce both namespace and pod label constraints simultaneously (AND logic).
- **Hints:** Put `namespaceSelector` and `podSelector` under the SAME array dash `-` in `from:`.
- **Expected Result:** Only matching pods can establish TCP connections on port 5432.
- **Verification:**

```
kubectl describe netpol allow-frontend-to-db -n backend-ns
```

- **Full Solution:**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-db
  namespace: backend-ns
spec:
  podSelector:
    matchLabels:
      app: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          env: prod
      podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 5432
```

```
kubectl create namespace frontend-ns
kubectl label namespace frontend-ns env=prod
kubectl create namespace backend-ns
kubectl apply -f netpol-db.yaml
```

- **Common Mistakes:** Putting `namespaceSelector` and `podSelector` on separate dashes, converting the rule into a loose OR condition.
- **Troubleshooting:** Verify namespace labels with `kubectl get ns --show-labels`.
- **Cleanup Instructions:**

```
kubectl delete namespace frontend-ns backend-ns
```

LAB 43: Restricting Outbound Egress with Port 53 DNS Whitelisting

- **Difficulty:** Advanced
- **Estimated Duration:** 12 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Restrict egress traffic for secure pods while explicitly allowing UDP/TCP port 53 to CoreDNS to prevent broken name resolution.
- **Prerequisites:** NetworkPolicy capable CNI.
- **Cluster Environment:** Standard cluster.
- **Initial State:** Namespace `restricted-apps`.

- **Task Description:**
    1. Create namespace `restricted-apps`.
    2. Deploy Pod `secure-worker` in `restricted-apps`.
    3. Author a NetworkPolicy `restrict-egress` that:
        - Applies to all pods in `restricted-apps`.
        - Allows egress strictly to IP range `10.0.0.0/16`.
        - Explicitly permits egress to CoreDNS on port 53 (UDP and TCP).
    4. Verify that DNS resolution continues to work inside `secure-worker`.
- **Constraints:** Do not use `policyTypes: [Egress]` without port 53 allowance.
- **Hints:** Allow egress to `namespaceSelector: {}` on port 53 to reach `kube-system` CoreDNS.
- **Expected Result:** `nslookup` succeeds inside `secure-worker`, but outbound curls to public internet fail.
- **Verification:**

```
kubectl exec -it secure-worker -n restricted-apps -- nslookup kubernetes.default
# Expected: Resolves successfully
```

- **Full Solution:**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: restrict-egress
  namespace: restricted-apps
spec:
  podSelector: {}
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.0/16
  - to:
    - namespaceSelector: {}
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
```

```
kubectl create namespace restricted-apps
kubectl run secure-worker --image=busybox:1.36 -n restricted-apps -- sleep 3600
kubectl apply -f egress-netpol.yaml
```

- **Common Mistakes:** Forgetting that CoreDNS also listens on TCP port 53 for large query responses.
- **Troubleshooting:** If DNS fails, test by curling the CoreDNS ClusterIP directly to see if packets are dropped.

- **Cleanup Instructions:**

```
kubectl delete namespace restricted-apps
```

LAB 44: External IP CIDR Ingress Filtering using `ipBlock`

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Author an `ipBlock` NetworkPolicy rule allowing traffic from a corporate subnet while carving out an exception CIDR block.
- **Prerequisites:** NetworkPolicy capable CNI.
- **Cluster Environment:** Any cluster.
- **Initial State:** Namespace `corporate-network`.
- **Task Description:**
    1. Create namespace `corporate-network`.
    2. Deploy Pod `internal-api` in `corporate-network` listening on port 80.
    3. Create NetworkPolicy `allow-corp-cidr` selecting `internal-api`:
        - Allow Ingress from CIDR `192.168.0.0/16`.
        - Except sub-block `192.168.100.0/24` (untrusted DMZ).
        - Port: TCP 80.
    4. Inspect the resulting policy with `kubectl describe`.
- **Constraints:** Use `ipBlock` with `except`.
- **Hints:** `except` must be an array of CIDRs contained within the parent `cidr`.
- **Expected Result:** Policy is accepted and displays allowed CIDRs and excluded subnets.
- **Verification:**

```
kubectl describe netpol allow-corp-cidr -n corporate-network
```

- **Full Solution:**

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-corp-cidr
  namespace: corporate-network
spec:
  podSelector:
    matchLabels:
      app: internal-api
  policyTypes:
  - Ingress
  ingress:
  - from:
    - ipBlock:
        cidr: 192.168.0.0/16
        except:
        - 192.168.100.0/24
    ports:
    - protocol: TCP
      port: 80
```

```
kubectl create namespace corporate-network
kubectl run internal-api --image=nginx:alpine -n corporate-network -l app=internal-api
kubectl apply -f ipblock-netpol.yaml
```

- **Common Mistakes:** Placing an `except` CIDR that is not a mathematical subnet of the outer `cidr`.
- **Troubleshooting:** Validate YAML syntax with `kubectl apply --dry-run=client -f ipblock-netpol.yaml`.
- **Cleanup Instructions:**

```
kubectl delete namespace corporate-network
```

LAB 45: Live Packet Tracing with Ephemeral Debug Pods and Netshoot

- **Difficulty:** Advanced
- **Estimated Duration:** 12 minutes
- **Domain:** Services & Networking (20%)
- **Learning Objective:** Deploy an ephemeral troubleshooting container (`nicolaka/netshoot`) into a target Pod's network namespace to trace live TCP packets with `tcpdump`.
- **Prerequisites:** Cluster with EphemeralContainers enabled (standard in v1.25+).
- **Cluster Environment:** Any cluster.
- **Initial State:** Running Pod `web-target` in `default`.
- **Task Description:**
    1. Deploy Pod `web-target` running `nginx:1.27`.

2. Use `kubectl debug` to attach an ephemeral container running image `nicolaka/netshoot` into `web-target` sharing its network namespace.
3. Inside the debug container, run `tcpdump -i eth0 -nn -s0 -v port 80`.
4. From another terminal, generate traffic to `web-target`.
5. Observe the live SYN, SYN-ACK, and ACK handshake in the tcpdump output.

- **Constraints:** Do not modify the original Pod specification or restart the container.
- **Hints:** Use `kubectl debug -it web-target --image=nicolaka/netshoot --target=web-target`.
- **Expected Result:** Live packet capture displays TCP conversation streams.
- **Verification:**

```
# Observed output inside debug container:
# IP client.38290 > web-target.80: Flags [S], seq ...
```

- **Full Solution:**

```
kubectl run web-target --image=nginx:1.27
# In Terminal 1:
kubectl debug -it web-target --image=nicolaka/netshoot -- /bin/bash
# Inside netshoot shell:
tcpdump -i any port 80 -n

# In Terminal 2:
POD_IP=$(kubectl get pod web-target -o jsonpath='{.status.podIP}')
kubectl run temp-curl --image=curlimages/curl --rm -it --restart=Never -- http://${POD_IP}
```

- **Common Mistakes:** Omitting the ephemeral container image, defaulting to alpine without networking tools.
- **Troubleshooting:** If `kubectl debug` fails, check if the cluster has RBAC permission for subresource `pods/ephemeralcontainers`.
- **Cleanup Instructions:**

```
kubectl delete pod web-target
```

VOLUME II: THE CKA PRACTICAL WORKBOOK

DOMAIN 4: STORAGE (LABS 46–53)

LAB 46: Static PersistentVolume and PVC 1-to-1 Binding Lifecycle

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Storage (10%)
- **Learning Objective:** Author a static `PersistentVolume` and a matching `PersistentVolumeClaim`, observing exclusive 1-to-1 binding and capacity allocation.
- **Prerequisites:** Running cluster with host filesystem access.
- **Cluster Environment:** Standard Kubernetes cluster.
- **Initial State:** Directory `/mnt/data/lab46` created on host node.
- **Task Description:**
    1. Author a PersistentVolume named `pv-static-10g`:
        - Capacity: `10Gi`
        - AccessMode: `ReadWriteOnce`
        - ReclaimPolicy: `Retain`
        - StorageClassName: `manual-storage`
        - Type: `hostPath` at path `/mnt/data/lab46`
    2. Author a PersistentVolumeClaim named `pvc-claim-5g` in `default`:
        - Requested Storage: `5Gi`
        - AccessMode: `ReadWriteOnce`
        - StorageClassName: `manual-storage`
    3. Verify that `pvc-claim-5g` binds to `pv-static-10g`.
    4. Note that the remaining 5Gi on the PV is locked and unavailable to other claims.
- **Constraints:** Do not use default storage classes.
- **Hints:** A PVC can bind to a PV of equal or greater capacity.
- **Expected Result:** Both PV and PVC show status `Bound`.
- **Verification:**

```
kubectl get pv pv-static-10g
kubectl get pvc pvc-claim-5g
```

- **Full Solution:**

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-static-10g
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual-storage
  hostPath:
    path: /mnt/data/lab46
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-claim-5g
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: manual-storage
  resources:
    requests:
      storage: 5Gi
```

```
kubectl apply -f static-pv-pvc.yaml
```

- **Common Mistakes:** Mismatching `storageClassName` between PV and PVC, causing PVC to remain in `Pending`.
- **Troubleshooting:** Run `kubectl describe pvc pvc-claim-5g` to inspect binding events.
- **Cleanup Instructions:**

```
kubectl delete pvc pvc-claim-5g
kubectl delete pv pv-static-10g
```

LAB 47: Dynamic Volume Provisioning with StorageClasses

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Storage (10%)

- **Learning Objective:** Configure a custom `StorageClass` with dynamic provisioning and observe automated PV creation upon PVC submission.
- **Prerequisites:** Cluster with dynamic provisioner (e.g. `rancher.io/local-path` or cloud provisioner).
- **Cluster Environment:** Standard cluster.
- **Initial State:** No StorageClass named `fast-dynamic`.
- **Task Description:**
    1. Create a `StorageClass` named `fast-dynamic` utilizing provisioner `rancher.io/local-path` (or target cluster CSI provisioner).
    2. Set `reclaimPolicy: Delete` and `volumeBindingMode: Immediate`.
    3. Create a PVC `dynamic-claim` in namespace `default` requesting `2Gi` on `fast-dynamic`.
    4. Verify that Kubernetes dynamically generates a new `PersistentVolume` without manual administrator intervention.
    5. Deploy a Pod `data-writer` mounting the claim at `/data` and write a test file.
- **Constraints:** Must use dynamic provisioning.
- **Hints:** Look at existing StorageClasses with `kubectl get sc` to find the active provisioner string.
- **Expected Result:** PVC immediately binds to an auto-created PV named `pvc-<uuid>`.
- **Verification:**

```
kubectl get pvc dynamic-claim
kubectl get pv | grep dynamic-claim
```

- **Full Solution:**

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-dynamic
provisioner: rancher.io/local-path
reclaimPolicy: Delete
volumeBindingMode: Immediate
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: dynamic-claim
spec:
  storageClassName: fast-dynamic
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: data-writer
spec:
  containers:
  - name: writer
    image: alpine
    command: ["/bin/sh", "-c", "echo 'storage verified' > /data/test.txt; sleep 3600"]
    volumeMounts:
    - name: storage
      mountPath: /data
  volumes:
  - name: storage
    persistentVolumeClaim:
      claimName: dynamic-claim
```

```
kubectl apply -f dynamic-storage.yaml
```

- **Common Mistakes:** Typo in `provisioner` name causing provisioner lookup failures.
- **Troubleshooting:** Run `kubectl describe pvc dynamic-claim` to check `ExternalProvisioning` events.
- **Cleanup Instructions:**

```
kubectl delete -f dynamic-storage.yaml
```

LAB 48: Delayed Volume Binding with `WaitForFirstConsumer`

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Storage (10%)

- **Learning Objective:** Configure a StorageClass with `volumeBindingMode: WaitForFirstConsumer` to ensure volumes are provisioned in the exact availability zone of the scheduled Pod.
- **Prerequisites:** Multi-node cluster.
- **Cluster Environment:** Multi-worker cluster.
- **Initial State:** StorageClass `topology-aware-sc`.
- **Task Description:**
    1. Author a StorageClass `delayed-sc` with `volumeBindingMode: WaitForFirstConsumer`.
    2. Create PVC `delayed-pvc` requesting 1Gi on `delayed-sc`.
    3. Observe that `delayed-pvc` remains in `Pending` with message `waiting for first consumer to be created before binding`.
    4. Create Pod `delayed-consumer` targeting node `worker-2` with `nodeName: worker-2` that mounts `delayed-pvc`.
    5. Verify that as soon as the Pod is created, the volume binds immediately on `worker-2`.
- **Constraints:** Understand why `Pending` status is normal before Pod scheduling.
- **Hints:** The volume binding controller waits for the scheduler to populate `spec.nodeName`.
- **Expected Result:** PVC transitions from `Pending` to `Bound` once consumer Pod is scheduled.
- **Verification:**

```
kubectl get pvc delayed-pvc
# Should show Bound only after Pod is applied
```

- **Full Solution:**

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: delayed-sc
provisioner: rancher.io/local-path
volumeBindingMode: WaitForFirstConsumer
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: delayed-pvc
spec:
  storageClassName: delayed-sc
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: delayed-consumer
spec:
  containers:
  - name: app
    image: alpine
    command: ["sleep", "3600"]
    volumeMounts:
    - name: data
      mountPath: /data
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: delayed-pvc
```

```
kubectl apply -f delayed-sc-pvc.yaml
# Verify PVC is Pending
kubectl apply -f delayed-pod.yaml
# Verify PVC is Bound
```

- **Common Mistakes:** Panicking when the PVC shows `Pending` before any Pod has referenced it.
- **Troubleshooting:** Run `kubectl describe pvc delayed-pvc` to confirm message `WaitForFirstConsumer`.
- **Cleanup Instructions:**

```
kubectl delete pod delayed-consumer
kubectl delete pvc delayed-pvc
kubectl delete sc delayed-sc
```

LAB 49: Reclaim Policy Enforcement (`Retain` vs `Delete`) and Released Recovery

- **Difficulty:** Advanced
- **Estimated Duration:** 15 minutes
- **Domain:** Storage (10%)
- **Learning Objective:** Recover data from a `Released` PersistentVolume by manually stripping `spec.claimRef` and re-binding it to a new claim.
- **Prerequisites:** Working cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** PV `pv-reclaim-test` with `persistentVolumeReclaimPolicy: Retain`.
- **Task Description:**
    1. Create a PV `pv-reclaim-test` with capacity 2Gi and `reclaimPolicy: Retain`.
    2. Bind it with PVC `pvc-original`.
    3. Write data to the volume from a temporary pod.
    4. Delete `pvc-original`.
    5. Observe that `pv-reclaim-test` enters `Released` status and refuses to bind to a new claim `pvc-new`.
    6. Manually edit `pv-reclaim-test` to delete `spec.claimRef`.
    7. Verify status changes to `Available` and `pvc-new` successfully binds to it with original data intact.
- **Constraints:** Do not delete or recreate the PersistentVolume object.
- **Hints:** Use `kubectl edit pv pv-reclaim-test` and remove the `claimRef` block.
- **Expected Result:** PV transitions `Bound` $\to$ `Released` $\to$ `Available` $\to$ `Bound`.
- **Verification:**

```
kubectl get pv pv-reclaim-test
```

- **Full Solution:**

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-reclaim-test
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual-reclaim
  hostPath:
    path: /tmp/reclaim-data
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-original
spec:
  storageClassName: manual-reclaim
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
```

```
kubectl apply -f pv-reclaim.yaml
# Delete PVC
kubectl delete pvc pvc-original
# PV is now Released
kubectl patch pv pv-reclaim-test -p '{"spec":{"claimRef": null}}'
# PV is now Available!
```

- **Common Mistakes:** Trying to re-bind without removing `claimRef`, causing new PVC to hang in `Pending`.
- **Troubleshooting:** Inspect `kubectl get pv pv-reclaim-test -o yaml` to verify `claimRef` is completely absent.
- **Cleanup Instructions:**

```
kubectl delete pv pv-reclaim-test
```

LAB 50: Single-Pod Data Locking with `ReadWriteOncePod` (RWOP)

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Storage (10%)
- **Learning Objective:** Enforce strict single-pod data access using modern access mode `ReadWriteOncePod` introduced in Kubernetes v1.29+.

- **Prerequisites:** Kubernetes v1.29+ cluster with CSI supporting RWOP.
- **Cluster Environment:** Modern cluster.
- **Initial State:** Namespace `rwop-demo`.
- **Task Description:**
    1. Create namespace `rwop-demo`.
    2. Author a PVC `exclusive-pvc` requesting 1Gi with accessMode `ReadWriteOncePod`.
    3. Deploy Pod `first-pod` mounting `exclusive-pvc`.
    4. Attempt to deploy a second Pod `second-pod` mounting the exact same `exclusive-pvc` on the same node.
    5. Observe that `second-pod` blocks in `Pending` / `ContainerCreating` with a volume exclusivity conflict error.
- **Constraints:** Access mode must be strictly `ReadWriteOncePod`.
- **Hints:** In standard `ReadWriteOnce`, multiple pods on the *same node* can mount the volume. `ReadWriteOncePod` forbids this!
- **Expected Result:** Second pod cannot mount the volume while the first pod is active.
- **Verification:**

```
kubectl describe pod second-pod -n rwop-demo | grep -i failedmount
```

- **Full Solution:**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: exclusive-pvc
  namespace: rwop-demo
spec:
  accessModes:
    - ReadWriteOncePod
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: first-pod
  namespace: rwop-demo
spec:
  containers:
  - name: c1
    image: alpine
    command: ["sleep", "3600"]
    volumeMounts:
    - name: vol
      mountPath: /data
  volumes:
  - name: vol
    persistentVolumeClaim:
      claimName: exclusive-pvc
---
apiVersion: v1
kind: Pod
metadata:
  name: second-pod
  namespace: rwop-demo
spec:
  containers:
  - name: c2
    image: alpine
    command: ["sleep", "3600"]
    volumeMounts:
    - name: vol
      mountPath: /data
  volumes:
  - name: vol
    persistentVolumeClaim:
      claimName: exclusive-pvc
```

```
kubectl create namespace rwop-demo
kubectl apply -f rwop-demo.yaml
```

- **Common Mistakes:** Confusing `ReadWriteOnce` with `ReadWriteOncePod`.
- **Troubleshooting:** Run `kubectl get events -n rwop-demo` to see volume lock rejection.
- **Cleanup Instructions:**

```
kubectl delete namespace rwop-demo
```

LAB 51: Dynamic PVC Online Volume Expansion

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Storage (10%)
- **Learning Objective:** Perform an online volume expansion of an active PersistentVolumeClaim from 1Gi to 3Gi without restarting application Pods.
- **Prerequisites:** StorageClass with `allowVolumeExpansion: true`.
- **Cluster Environment:** Dynamic storage cluster.
- **Initial State:** PVC `expandable-pvc` currently bound at 1Gi.
- **Task Description:**
    1. Verify the StorageClass allows volume expansion (`allowVolumeExpansion: true`).
    2. Deploy Pod `app-resizing` mounting `expandable-pvc` at `/data`.
    3. Edit `expandable-pvc` using `kubectl patch` to increase requested storage to `3Gi`.
    4. Verify the PVC status reflects `3Gi` and the container filesystem expands online.
- **Constraints:** Do not delete or terminate `app-resizing`.
- **Hints:** You can only expand storage size; shrinking storage is strictly prohibited by Kubernetes.
- **Expected Result:** Filesystem expands seamlessly without downtime.
- **Verification:**

```
kubectl get pvc expandable-pvc
# Capacity should show 3Gi
kubectl exec -it app-resizing -- df -h /data
```

- **Full Solution:**

```
kubectl patch pvc expandable-pvc -p '{"spec":{"resources":{"requests":{"storage":"3Gi"}}}}'
```

- **Common Mistakes:** Attempting to reduce volume size (e.g. from 3Gi to 1Gi), which is rejected with 422 Unprocessable Entity.
- **Troubleshooting:** If capacity does not change, check if the StorageClass has `allowVolumeExpansion: true`.
- **Cleanup Instructions:**

```
kubectl delete pod app-resizing
kubectl delete pvc expandable-pvc
```

LAB 52: Multi-Pod Shared Read-Write Storage (`ReadWriteMany`)

- **Difficulty:** Intermediate
- **Estimated Duration:** 12 minutes
- **Domain:** Storage (10%)
- **Learning Objective:** Configure a shared volume with `ReadWriteMany` access mode and verify concurrent read-write access from pods on different nodes.
- **Prerequisites:** Clustered/shared storage provisioner (NFS or local-path shared).
- **Cluster Environment:** Multi-worker cluster.
- **Initial State:** Namespace `shared-storage`.
- **Task Description:**
    1. Create namespace `shared-storage`.
    2. Create PVC `shared-pvc` with `accessModes: [ReadWriteMany]` requesting 2Gi.
    3. Deploy Pod `writer-pod` on `worker-1` mounting `shared-pvc` and writing timestamps to `/shared/log.txt`.
    4. Deploy Pod `reader-pod` on `worker-2` mounting `shared-pvc` and tailing `/shared/log.txt`.
    5. Verify `reader-pod` sees live updates written by `writer-pod`.
- **Constraints:** Must use `ReadWriteMany`.
- **Hints:** Block storage (AWS EBS, GCP PD) does NOT support RWX. File storage (NFS, EFS, CephFS) is required.
- **Expected Result:** Concurrent multi-node read/write succeeds.
- **Verification:**

```
kubectl logs reader-pod -n shared-storage
```

- **Full Solution:**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: shared-pvc
  namespace: shared-storage
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 2Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: writer-pod
  namespace: shared-storage
spec:
  nodeName: worker-1
  containers:
  - name: writer
    image: alpine
    command: ["/bin/sh", "-c", "while true; do date >> /shared/log.txt; sleep 2; done"]
    volumeMounts:
    - name: shared-vol
      mountPath: /shared
  volumes:
  - name: shared-vol
    persistentVolumeClaim:
      claimName: shared-pvc
---
apiVersion: v1
kind: Pod
metadata:
  name: reader-pod
  namespace: shared-storage
spec:
  nodeName: worker-2
  containers:
  - name: reader
    image: alpine
    command: ["/bin/sh", "-c", "sleep 5; cat /shared/log.txt; sleep 3600"]
    volumeMounts:
    - name: shared-vol
      mountPath: /shared
  volumes:
  - name: shared-vol
    persistentVolumeClaim:
      claimName: shared-pvc
```

```
kubectl create namespace shared-storage
kubectl apply -f shared-storage.yaml
```

- **Common Mistakes:** Requesting `ReadWriteMany` against an EBS/GCP PD CSI driver that only supports `ReadWriteOnce`.

- **Troubleshooting:** Check `kubectl describe pvc shared-pvc -n shared-storage` for unsupported access mode errors.
- **Cleanup Instructions:**

```
kubectl delete namespace shared-storage
```

LAB 53: CSI Driver Inspection and VolumeAttachment Triage

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Storage (10%)
- **Learning Objective:** Inspect Container Storage Interface (CSI) node registration, volume drivers, and `VolumeAttachment` API objects.
- **Prerequisites:** CSI-enabled cluster.
- **Cluster Environment:** Standard Kubernetes cluster.
- **Initial State:** Running cluster with active workloads.
- **Task Description:**
  1. Inspect all registered CSI drivers using `kubectl get csidrivers`.
  2. Inspect CSI node registrations across all nodes using `kubectl get csinodes`.
  3. Inspect active `VolumeAttachment` objects across the cluster.
  4. Identify which node has attached volume `pvc-xxxx`.
- **Constraints:** Understand how the CSI Controller attaches volumes before the CSI Node mounts them.
- **Hints:** `VolumeAttachment` objects are cluster-scoped.
- **Expected Result:** Full mapping of PVs to physical nodes via `VolumeAttachment`.
- **Verification:**

```
kubectl get csidrivers
kubectl get csinodes -o wide
kubectl get volumeattachments
```

- **Full Solution:**

```
kubectl get csidrivers -o custom-columns=NAME:.metadata.name,ATTACH-REQUIRED:.spec.attachRequired,P
kubectl get csinodes
kubectl get volumeattachment -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName,PV:.spec.sou
```

- **Common Mistakes:** Looking for `VolumeAttachment` in a specific namespace (it is non-namespaced).
- **Troubleshooting:** If a volume attachment hangs, describe the `VolumeAttachment` object to view CSI driver attach errors.
- **Cleanup Instructions:** None.

VOLUME II: THE CKA PRACTICAL WORKBOOK

DOMAIN 5: TROUBLESHOOTING (LABS 54–66)

LAB 54: Pod Stuck in `Pending` (Resource Depletion and Taints)

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Diagnose why a Pod remains in `Pending` status by inspecting scheduler events and correcting unschedulable constraints.
- **Prerequisites:** Running cluster.
- **Cluster Environment:** Standard Kubernetes cluster.
- **Initial State:** Pod `batch-calc` created in `default` stuck in `Pending`.
- **Task Description:**
    1. Inspect the cluster to locate a pod named `batch-calc` stuck in `Pending`.
    2. Inspect the pod's events to identify why the scheduler failed to place it.
    3. You will discover that the container requested `cpu: "64"`, which exceeds cluster capacity.
    4. Fix the manifest to request `cpu: "100m"`, re-apply, and verify the Pod enters `Running` status.
- **Constraints:** Do not delete or cordon worker nodes.
- **Hints:** Look at the bottom of `kubectl describe pod batch-calc`.
- **Expected Result:** Pod transitions from `Pending` to `Running`.
- **Verification:**

```
kubectl get pod batch-calc
# STATUS: Running
```

- **Full Solution:**

```
# 1. Diagnose
kubectl describe pod batch-calc
# Event: 0/3 nodes are available: 3 Insufficient cpu.

# 2. Extract and edit
kubectl get pod batch-calc -o yaml > /tmp/batch-calc.yaml
sed -i 's/cpu: "64"/cpu: 100m/g' /tmp/batch-calc.yaml

# 3. Replace
kubectl replace --force -f /tmp/batch-calc.yaml
```

- **Common Mistakes:** Trying to edit `requests.cpu` on a running pod with `kubectl edit` without `--force` (requests are immutable on running pods in standard clusters).
- **Troubleshooting:** Run `kubectl get events --field-selector involvedObject.name=batch-calc`.
- **Cleanup Instructions:**

```
kubectl delete pod batch-calc
```

LAB 55: Diagnosing and Fixing `CrashLoopBackOff`

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Triage a container that boots and immediately crashes using `kubectl logs --previous` and repair application startup arguments.
- **Prerequisites:** Running cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Pod `log-forwarder` in namespace `logging` in `CrashLoopBackOff`.
- **Task Description:**
  1. Inspect Pod `log-forwarder` in namespace `logging`.
  2. Read the crash logs using `kubectl logs log-forwarder -n logging --previous`.
  3. Identify that the command failed with error: `cat: can't open '/etc/config/app.conf': No such file or directory`.
  4. Create a ConfigMap named `forwarder-config` containing key `app.conf` with text `log_level=debug`.
  5. Mount this ConfigMap at `/etc/config` in the Pod manifest and re-deploy.
  6. Verify the Pod reaches `Running` status with zero restarts.
- **Constraints:** Do not alter the container entrypoint command.
- **Hints:** Use `--previous` to see logs from the crashed container before the restart backoff.
- **Expected Result:** Pod stabilizes at `Running`.
- **Verification:**

```
kubectl get pod log-forwarder -n logging
```

- **Full Solution:**

```
kubectl create namespace logging --dry-run=client -o yaml | kubectl apply -f -

# Check previous log
kubectl logs log-forwarder -n logging --previous

# Create missing config
kubectl create configmap forwarder-config --from-literal=app.conf="log_level=debug" -n logging

# Update Pod manifest
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: log-forwarder
  namespace: logging
spec:
  containers:
  - name: forwarder
    image: busybox:1.36
    command: ["/bin/sh", "-c", "cat /etc/config/app.conf && sleep 3600"]
    volumeMounts:
    - name: cfg
      mountPath: /etc/config
  volumes:
  - name: cfg
    configMap:
      name: forwarder-config
EOF
```

- **Common Mistakes:** Running `kubectl logs` without `--previous`, which often displays an empty log if the container is currently in the backoff sleep cycle.
- **Troubleshooting:** Inspect exit code in `kubectl describe pod log-forwarder -n logging`.
- **Cleanup Instructions:**

```
kubectl delete namespace logging
```

LAB 56: Fixing `ImagePullBackOff` and Private Registry Secrets

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Identify image name typographical errors and configure `imagePullSecrets` for private container registries.
- **Prerequisites:** Running cluster.

- **Cluster Environment:** Any cluster.
- **Initial State:** Pod `secure-app` showing `ImagePullBackOff`.
- **Task Description:**
    1. Inspect Pod `secure-app` in `default`.
    2. Run `kubectl describe pod secure-app` to identify the image error.
    3. Notice the typo in the image name: `nginxx:1.27` instead of `nginx:1.27`.
    4. Update the pod image to `nginx:1.27`.
    5. Verify the pod transitions from `ImagePullBackOff` $\to$ `ContainerCreating` $\to$ `Running`.
- **Constraints:** Do not recreate the pod if you can patch it cleanly, or replace it rapidly.
- **Hints:** In a Pod, `spec.containers[0].image` can be edited, or recreate with `--force`.
- **Expected Result:** Pod pulls successfully and runs.
- **Verification:**

```
kubectl get pod secure-app
```

- **Full Solution:**

```
kubectl describe pod secure-app | grep -A 3 Events
kubectl set image pod secure-app secure-app=nginx:1.27
```

- **Common Mistakes:** Confusing `ImagePullBackOff` (image not found) with `CrashLoopBackOff` (image downloaded, but binary failed).
- **Troubleshooting:** Inspect exact event string in `kubectl describe pod secure-app`.
- **Cleanup Instructions:**

```
kubectl delete pod secure-app
```

LAB 57: Out-Of-Memory (`OOMKilled` Exit Code 137) Limit Tuning

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Diagnose an application terminated with Exit Code 137 (OOMKilled) and calibrate memory limits to accommodate application heap spikes.
- **Prerequisites:** Running cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Deployment `cache-worker` whose pods restart every 30 seconds with Exit Code 137.
- **Task Description:**
    1. Inspect the pods of Deployment `cache-worker`.
    2. Notice the restart count incrementing and `Last State: Terminated, Reason: OOMKilled`.

3. Inspect container memory limits: currently set to `32Mi`.
4. Update the Deployment manifest to increase memory limits to `256Mi` and memory requests to `128Mi`.
5. Monitor the new rollout and verify the pods remain running with 0 restarts.

- **Constraints:** Understand why the Linux kernel OOM killer terminates processes exceeding `limits.memory`.
- **Hints:** Exit Code 137 $= 128 + 9$ (SIGKILL).
- **Expected Result:** Deployment pods stabilize at `Running` with zero subsequent restarts.
- **Verification:**

```
kubectl get pods -l app=cache-worker
```

- **Full Solution:**

```
kubectl describe pod -l app=cache-worker | grep -E "Reason|Exit Code"
kubectl set resources deployment cache-worker --limits=memory=256Mi --requests=memory=128Mi
kubectl rollout status deployment cache-worker
```

- **Common Mistakes:** Increasing CPU limits instead of memory limits (CPU limits cause throttling; only memory causes OOMKilled).
- **Troubleshooting:** Check node kernel dmesg for kernel OOM killer logs: `dmesg -T | grep -i oom`.
- **Cleanup Instructions:**

```
kubectl delete deployment cache-worker
```

LAB 58: Triage and Recovery of a `NotReady` Worker Node

- **Difficulty:** Advanced
- **Estimated Duration:** 15 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** SSH into a failed worker node, inspect systemd service failures, repair a dead `kubelet` daemon, and verify node restoration.
- **Prerequisites:** Multi-node cluster with SSH access to `worker-1`.
- **Cluster Environment:** `kubeadm` multi-node cluster.
- **Initial State:** `worker-1` shows `STATUS: NotReady`.
- **Task Description:**

1. On the control plane, run `kubectl get nodes` and observe `worker-1` is `NotReady`.
2. SSH into `worker-1`.
3. Inspect the status of the `kubelet` service: `sudo systemctl status kubelet`.
4. Notice the service is inactive (dead).
5. Attempt to start the service: `sudo systemctl start kubelet`.

6. Inspect `sudo journalctl -u kubelet -e` to find why it fails: `unable to load client CA file: stat /etc/kubernetes/pki/ca.crt: no such file or directory`.
7. Discover someone accidentally renamed `/etc/kubernetes/pki/ca.crt` to `/etc/kubernetes/pki/ca.crt.bak`.
8. Restore the CA certificate file, start and enable `kubelet`.
9. Exit to control plane and verify `worker-1` transitions back to `Ready`.

- **Constraints:** Do not reboot the host.
- **Hints:** Kubelet requires the cluster CA certificate to validate the API server.
- **Expected Result:** `worker-1` returns to `Ready` status.
- **Verification:**

```
kubectl get nodes worker-1
# STATUS: Ready
```

- **Full Solution:**

```
# On worker-1:
sudo systemctl status kubelet
sudo journalctl -u kubelet -n 30 --no-pager
# Identify missing ca.crt
sudo mv /etc/kubernetes/pki/ca.crt.bak /etc/kubernetes/pki/ca.crt
sudo systemctl restart kubelet
sudo systemctl is-active kubelet
# Output: active
exit

# On Control Plane:
kubectl get nodes worker-1
```

- **Common Mistakes:** Failing to inspect `journalctl -u kubelet` and guessing random configurations.
- **Troubleshooting:** Verify node conditions: `kubectl describe node worker-1 | grep Ready`.
- **Cleanup Instructions:** None.

LAB 59: Broken Kubelet Cgroup Driver Recovery

- **Difficulty:** Advanced
- **Estimated Duration:** 12 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Resolve a fatal cgroup driver mismatch between `containerd` and `kubelet` causing node boot loops.
- **Prerequisites:** SSH access to a worker node.
- **Cluster Environment:** Node running containerd.
- **Initial State:** Kubelet failing with `misconfigured cgroup driver`.
- **Task Description:**

1. On worker node `worker-2`, kubelet fails to start.
2. Inspect `sudo journalctl -u kubelet -e`.
3. Locate error: `failed to run Kubelet: misconfiguration: cgroup driver: "cgroupfs" is different from docker/containerd cgroup driver "systemd"`.
4. Edit `/var/lib/kubelet/config.yaml`.
5. Update `cgroupDriver: cgroupfs` to `cgroupDriver: systemd`.
6. Restart `kubelet` and verify it starts cleanly without errors.

- **Constraints:** Enforce `systemd` cgroup driver everywhere.
- **Hints:** Kubelet's configuration is managed via `/var/lib/kubelet/config.yaml`.
- **Expected Result:** Kubelet service enters `active (running)`.
- **Verification:**

```
sudo systemctl status kubelet
# Active: active (running)
```

- **Full Solution:**

```
# On worker-2:
sudo sed -i 's/cgroupDriver: cgroupfs/cgroupDriver: systemd/g' /var/lib/kubelet/config.yaml
sudo systemctl restart kubelet
sudo systemctl status kubelet --no-pager
```

- **Common Mistakes:** Modifying flags in `/etc/default/kubelet` that have been deprecated in modern Kubernetes. Edit `/var/lib/kubelet/config.yaml` directly.
- **Troubleshooting:** If kubelet still fails, verify `/etc/containerd/config.toml` has `SystemdCgroup = true`.
- **Cleanup Instructions:** None.

LAB 60: Broken Control Plane Static Pod Manifest Repair

- **Difficulty:** Advanced
- **Estimated Duration:** 15 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Fix an invalid CLI flag inside `/etc/kubernetes/manifests/kube-scheduler.yaml` that prevents the Kubernetes scheduler from starting.
- **Prerequisites:** SSH access to control plane node with sudo privileges.
- **Cluster Environment:** `kubeadm` control plane node.
- **Initial State:** All newly created pods stay in `Pending` forever; `kube-scheduler` static pod is not running.
- **Task Description:**

1. Notice that all pods across the cluster remain `Pending` even though nodes have ample CPU and RAM.

2. Check control plane pods: `kubectl get pods -n kube-system`. Notice `kube-scheduler` is missing or crashed.
3. SSH into the control plane node.
4. Inspect `/etc/kubernetes/manifests/kube-scheduler.yaml`.
5. Run `sudo crictl ps -a | grep scheduler` and check logs: `sudo crictl logs <id>`.
6. Discover someone introduced a bogus flag: `--port=10251` or `--unknown-flag=true`.
7. Remove the invalid flag from `/etc/kubernetes/manifests/kube-scheduler.yaml`.
8. Observe kubelet automatically detect the file change and relaunch the scheduler container within 10 seconds.
9. Verify pending pods are immediately scheduled and transition to `Running`.

- **Constraints:** Do not use `kubectl apply` on static pod manifests.
- **Hints:** Kubelet watches `/etc/kubernetes/manifests/` with inotify. Saving the file automatically triggers container recreation.
- **Expected Result:** `kube-scheduler` starts, and pending pods schedule immediately.
- **Verification:**

```
kubectl get pods -n kube-system | grep scheduler
kubectl get pods
```

- **Full Solution:**

```
# On Control Plane:
sudo crictl ps -a --name=kube-scheduler
# Inspect logs of exited container
sudo crictl logs $(sudo crictl ps -a --name=kube-scheduler -q | head -n 1)

# Edit manifest and remove invalid line
sudo nano /etc/kubernetes/manifests/kube-scheduler.yaml

# Wait 10 seconds and verify
kubectl get pods -n kube-system -l component=kube-scheduler
```

- **Common Mistakes:** Trying to delete the static pod with `kubectl delete pod`; kubelet will simply recreate the broken container again until the YAML on disk is fixed!
- **Troubleshooting:** Check `journalctl -u kubelet` if the static pod is not detected.
- **Cleanup Instructions:** None.

LAB 61: Expired Cluster Certificate Recovery via `kubeadm certs renew`

- **Difficulty:** Advanced
- **Estimated Duration:** 12 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Recover an unresponsive cluster whose control plane TLS certificates have expired.

- **Prerequisites:** SSH access to control plane node.
- **Cluster Environment:** `kubeadm` cluster.
- **Initial State:** `kubectl` returns `Unable to connect to the server: x509: certificate has expired`.
- **Task Description:**
    1. Run `kubectl get nodes` and observe the x509 certificate expired error.
    2. SSH into the control plane node.
    3. Check certificate status: `sudo kubeadm certs check-expiration`.
    4. Observe that `admin.conf`, `apiserver`, etc. are expired.
    5. Renew all certificates: `sudo kubeadm certs renew all`.
    6. Copy the renewed admin credentials to `$HOME/.kube/config`: `sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config && sudo chown $(id -u):$(id -g) $HOME/.kube/config`.
    7. Restart control plane static pods by moving manifests out to `/tmp/` and back in.
    8. Verify `kubectl get nodes` functions cleanly.
- **Constraints:** Renew all certificates using official `kubeadm` tooling.
- **Hints:** Remember to update `$HOME/.kube/config` with the new client cert embedded in `/etc/kubernetes/admin.conf`!
- **Expected Result:** Cluster responds cleanly to all `kubectl` commands.
- **Verification:**

```
kubectl get nodes
sudo kubeadm certs check-expiration
```

- **Full Solution:**

```
# 1. Renew
sudo kubeadm certs renew all

# 2. Update user kubeconfig
mkdir -p $HOME/.kube
sudo cp -f /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# 3. Restart static pods
sudo mv /etc/kubernetes/manifests/*.yaml /tmp/
sleep 5
sudo mv /tmp/*.yaml /etc/kubernetes/manifests/

# 4. Verify
sleep 15
kubectl get nodes
```

- **Common Mistakes:** Renewing certificates in `/etc/kubernetes/pki` but forgetting to update `$HOME/.kube/config`, leaving the user with an expired client cert!
- **Troubleshooting:** Run `openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -dates` to verify valid dates.
- **Cleanup Instructions:** None.

LAB 62: CoreDNS Loop Crash Recovery

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Diagnose and resolve a CoreDNS infinite loop crash caused by systemd-resolved stub resolvers in host `/etc/resolv.conf`.
- **Prerequisites:** Control plane access.
- **Cluster Environment:** Ubuntu nodes with systemd-resolved.
- **Initial State:** CoreDNS pods in `CrashLoopBackOff`.
- **Task Description:**
    1. Notice CoreDNS pods in `kube-system` crash-looping.
    2. Inspect logs: `kubectl logs -n kube-system -l k8s-app=kube-dns`.
    3. Observe error: `plugin/loop: Loop (127.0.0.1:53 -> :53) detected for zone "."`.
    4. Inspect the host `/etc/resolv.conf` and discover `nameserver 127.0.0.53`.
    5. Update the kubelet configuration `/var/lib/kubelet/config.yaml` to point `resolvConf` to `/run/systemd/resolve/resolv.conf` (which points to real upstream DNS).
    6. Restart kubelet: `sudo systemctl restart kubelet`.
    7. Delete crash-looping CoreDNS pods and verify the new pods enter `Running` status without loop errors.
- **Constraints:** Do not disable CoreDNS loop detection plugin.
- **Hints:** `/run/systemd/resolve/resolv.conf` contains upstream nameservers instead of the local stub resolver.
- **Expected Result:** CoreDNS pods stabilize at `Running`.
- **Verification:**

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
# STATUS: Running, RESTARTS: 0
```

- **Full Solution:**

```
# Check logs
kubectl logs -n kube-system -l k8s-app=kube-dns

# Configure kubelet to use real upstream resolv.conf
sudo sed -i 's|resolvConf: .*|resolvConf: /run/systemd/resolve/resolv.conf|g' /var/lib/kubelet/conf
sudo systemctl restart kubelet

# Delete old pods to pick up new config
kubectl delete pods -n kube-system -l k8s-app=kube-dns
```

- **Common Mistakes:** Manually removing the `loop` plugin from the Corefile without fixing the underlying upstream loop, leading to high CPU exhaustion.

- **Troubleshooting:** Check host nameserver: `cat /run/systemd/resolve/resolv.conf`.
- **Cleanup Instructions:** None.

LAB 63: Network Service Endpoint Disconnect

- **Difficulty:** Beginner
- **Estimated Duration:** 8 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Diagnose why a Kubernetes Service returns connection refused or empty responses by identifying selector label mismatches.
- **Prerequisites:** Running cluster.
- **Cluster Environment:** Any cluster.
- **Initial State:** Service `auth-api-svc` and Deployment `auth-api` running in `default`.
- **Task Description:**
    1. Service `auth-api-svc` is failing to route traffic to application pods.
    2. Run `kubectl get endpoints auth-api-svc` and observe `ENDPOINTS: <none>`.
    3. Inspect the selector of the Service and compare it with the labels of the Deployment pods.
    4. Notice the Service selector specifies `app: auth-service`, but the Pod template is labeled `app: auth-api`.
    5. Fix the Service selector to match `app: auth-api`.
    6. Verify `kubectl get endpoints auth-api-svc` now lists all pod IP addresses.
- **Constraints:** Do not delete the Deployment.
- **Hints:** A Service will only route traffic to Pods whose labels match its `spec.selector` exactly.
- **Expected Result:** Service endpoints are populated and internal traffic flows.
- **Verification:**

```
kubectl get endpoints auth-api-svc
# Should show IP:Port
```

- **Full Solution:**

```
kubectl get svc auth-api-svc -o yaml
kubectl get pods --show-labels

# Update selector
kubectl patch svc auth-api-svc -p '{"spec":{"selector":{"app":"auth-api"}}}'

# Verify
kubectl get endpoints auth-api-svc
```

- **Common Mistakes:** Editing the Deployment pod labels instead of the Service selector, which breaks other label-dependent controllers.
- **Troubleshooting:** Inspect selector with `kubectl describe svc auth-api-svc`.

- **Cleanup Instructions:**

```
kubectl delete svc auth-api-svc
```

LAB 64: Broken NetworkPolicy Triage (Blocked Cross-Namespace Traffic)

- **Difficulty:** Intermediate
- **Estimated Duration:** 10 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Discover and fix a restrictive NetworkPolicy that is silently dropping cross-namespace API calls.
- **Prerequisites:** NetworkPolicy capable cluster.
- **Cluster Environment:** Any cluster with Calico/Cilium.
- **Initial State:** Namespaces `frontend` and `backend`. Frontend cannot connect to `backend-svc`.
- **Task Description:**
    1. Pod in namespace `frontend` attempts to curl `http://backend-svc.backend:8080` but times out.
    2. Inspect the `backend` namespace for active NetworkPolicies: `kubectl get netpol -n backend`.
    3. Inspect policy `lockdown-backend`: notice it allows Ingress only from namespaces labeled `tier: web`.
    4. Check labels on namespace `frontend`: `kubectl get ns frontend --show-labels`. Notice the label is missing.
    5. Label namespace `frontend` with `tier: web`.
    6. Verify connectivity is immediately restored.
- **Constraints:** Do not delete or disable the NetworkPolicy.
- **Hints:** NetworkPolicies selecting namespaces match against labels on the `Namespace` object, not the Pod object!
- **Expected Result:** Connections from `frontend` to `backend` succeed.
- **Verification:**

```
kubectl run curl-test --image=curlimages/curl --rm -it --restart=Never -n frontend -- http://backer
# Expected: HTTP 200 OK
```

- **Full Solution:**

```
kubectl describe netpol lockdown-backend -n backend
kubectl label namespace frontend tier=web --overwrite
```

- **Common Mistakes:** Labeling the Pod in `frontend` instead of labeling the `Namespace` resource.

- **Troubleshooting:** Run `kubectl get netpol -A` to ensure no competing default-deny egress policies exist in `frontend`.
- **Cleanup Instructions:**

```
kubectl delete namespace frontend backend
```

LAB 65: etcd Quorum Loss and `mvcc: database space exceeded` Alarm

- **Difficulty:** Advanced
- **Estimated Duration:** 15 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Resolve a frozen etcd cluster locked in read-only mode by compacting historical revisions, defragmenting storage, and disarming alarms.
- **Prerequisites:** SSH access to control plane node.
- **Cluster Environment:** `kubeadm` cluster.
- **Initial State:** Cluster refuses write operations with `etcdserver: mvcc: database space exceeded`.
- **Task Description:**
  1. Attempt to create a pod: `kubectl run test --image=nginx`.
  2. Observe failure: `etcdserver: mvcc: database space exceeded`.
  3. SSH into control plane node.
  4. Query active alarms using `etcdctl alarm list`. Notice alarm `NOSPACE`.
  5. Query current etcd revision.
  6. Compact etcd history up to current revision.
  7. Defragment the database storage on disk using `etcdctl defrag`.
  8. Disarm the alarm using `etcdctl alarm disarm`.
  9. Verify write access is restored by successfully creating the test pod.
- **Constraints:** Do not wipe the database or delete `/var/lib/etcd`.
- **Hints:** Defrag alone will not clear the alarm; you must run `alarm disarm`.
- **Expected Result:** Alarm is cleared and cluster writes succeed.
- **Verification:**

```
sudo ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/ku
  alarm list
# Expected: empty
kubectl run test-verify --image=nginx:alpine
# Expected: pod created successfully
```

- **Full Solution:**

```
# 1. Check alarm
sudo ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/ku
  alarm list

# 2. Find revision
REV=$(sudo ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/ku
  endpoint status --write-out="json" | jq .[0].Status.header.revision)

# 3. Compact
sudo ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/ku
  compact $REV

# 4. Defrag
sudo ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/ku
  defrag

# 5. Disarm
sudo ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/ku
  alarm disarm
```

- **Common Mistakes:** Running `compact` but omitting `defrag`, which leaves host database file size unchanged.
- **Troubleshooting:** Check disk usage of `/var/lib/etcd/member/snap/db`.
- **Cleanup Instructions:**

```
kubectl delete pod test-verify || true
```

LAB 66: Integrated Multi-Failure Cluster Outage (The CKA Grand Challenge)

- **Difficulty:** Advanced
- **Estimated Duration:** 20 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Solve a compound, cascading failure involving a dead kubelet, corrupted static pod manifest, broken CoreDNS, and locked Service.
- **Prerequisites:** Full control plane and worker node SSH access.
- **Cluster Environment:** Multi-node `kubeadm` cluster.
- **Initial State:** Cluster completely down: `kubectl` connection refused.
- **Task Description:**

1. Triage Stage 1: Control plane is unreachable on port 6443. SSH to control plane. Kubelet is inactive because Linux swap was enabled. Disable swap and start kubelet.
2. Triage Stage 2: Kubelet is running, but API Server static pod still fails to start. Inspect `/etc/kubernetes/manifests/kube-apiserver.yaml`. Fix a typo in `--etcd-servers=https://127.0.0.1:23799` (extra 9 on port). API server boots!
3. Triage Stage 3: `worker-1` shows `NotReady`. SSH to `worker-1`. Container runtime is stopped. Start `containerd` and `kubelet`. Node transitions to `Ready`.
4. Triage Stage 4: Workload Pods cannot resolve external or internal services. Inspect `kube-system` CoreDNS pods; they are running, but Service `kube-dns` has an incorrect targetPort (`targetPort: 5353` instead of `53`). Fix targetPort.
5. Run end-to-end verification across the entire cluster.

- **Constraints:** Solve each layer sequentially without rebooting nodes.
- **Hints:** Follow the 6-layer diagnostic hierarchy (Hardware $\to$ Runtime $\to$ Kubelet $\to$ Static Pods $\to$ Networking $\to$ App).
- **Expected Result:** All nodes `Ready`, all control plane components healthy, DNS resolves, and workloads serve traffic.
- **Verification:**

```
kubectl get nodes
kubectl get pods -A
kubectl run grand-check --image=busybox:1.36 --rm -it --restart=Never -- nslookup kubernetes.defaul
```

- **Full Solution:**

```
# Step 1: On Control Plane - Fix Swap
sudo swapoff -a
sudo systemctl restart kubelet

# Step 2: Fix API Server port typo
sudo sed -i 's/23799/2379/g' /etc/kubernetes/manifests/kube-apiserver.yaml
sleep 15

# Step 3: On worker-1 - Fix Containerd
ssh worker-1
sudo systemctl restart containerd
sudo systemctl restart kubelet
exit

# Step 4: Fix CoreDNS Service port
kubectl patch svc kube-dns -n kube-system --type='json' -p='[{"op": "replace", "path": "/spec/ports

# Step 5: Verify
kubectl get nodes
```

- **Common Mistakes:** Giving up at Step 1 or assuming the cluster is irreparably corrupted.
- **Troubleshooting:** Keep checking component health via `kubectl get --raw='/readyz?verbose'`.
- **Cleanup Instructions:** None (cluster is restored to health).

LAB 67: Advanced Chaos Recovery: Cross-Namespace Gateway API Failure, Missing ReferenceGrant & Kubelet Socket Mismatch

- **Difficulty:** Advanced
- **Estimated Duration:** 20 minutes
- **Domain:** Troubleshooting (30%)
- **Learning Objective:** Troubleshoot an advanced multi-tier infrastructure failure: a worker node reporting `NotReady` due to a misplaced CRI socket, combined with a detached cross-namespace Gateway API `HTTPRoute` failing traffic due to a missing `ReferenceGrant`.
- **Prerequisites:** Gateway API CRDs v1.1+ installed, multi-node cluster.
- **Cluster Environment:** Multi-node cluster with `infra-cluster` context.
- **Initial State:**
    1. Worker node `worker-2` is in `NotReady` condition.
    2. Namespace `gateway-infra` has Gateway `prod-gw` listening on HTTP port 80.
    3. Namespace `checkout-app` has Deployment `checkout-web` and Service `checkout-svc`.
    4. Namespace `checkout-app` has HTTPRoute `checkout-route` targeting `prod-gw`, but traffic to `checkout.example.com` returns 404 or connection refused.
- **Task Description:**
    1. Diagnose why `worker-2` is `NotReady`. SSH to `worker-2`, inspect `kubelet` journal logs, identify the misconfigured CRI socket path in `/var/lib/kubelet/kubeadm-flags.env` or `/var/lib/kubelet/config.yaml`, fix it, and restore the node to `Ready`.
    2. Inspect the condition of `HTTPRoute` `checkout-route` in namespace `checkout-app`.
    3. Notice that `checkout-route` has condition `Accepted: False` with reason `RefNotPermitted` because the parent Gateway is in `gateway-infra` while the route is in `checkout-app`.
    4. Author and apply a `ReferenceGrant` in namespace `gateway-infra` allowing HTTPRoutes from namespace `checkout-app` to bind to Gateway `prod-gw`.
    5. Verify `HTTPRoute` transitions to `Accepted: True` and traffic routes successfully to `checkout-svc`.
- **Constraints:** Do not delete or recreate the Gateway or worker node. Resolve root causes in-place.
- **Hints:** Use `kubectl describe httproute checkout-route -n checkout-app` to inspect conditions. Check `kubectl get referencegrant -A`.
- **Expected Result:** `worker-2` is `Ready`, and `checkout-route` is accepted and programmed.
- **Verification:**

```
kubectl get nodes worker-2
# Expected: Ready
kubectl get httproute checkout-route -n checkout-app -o jsonpath='{.status.parents[0].conditions[?(
# Expected: True
```

- **Full Solution:**

```
# Step 1: Fix worker-2 Kubelet CRI socket
ssh worker-2
sudo sed -i 's|/run/containerd/wrong.sock|/run/containerd/containerd.sock|g' /var/lib/kubelet/kubea
sudo systemctl daemon-reload
sudo systemctl restart kubelet
exit

# Step 2: Inspect HTTPRoute status
kubectl describe httproute checkout-route -n checkout-app

# Step 3: Create ReferenceGrant in gateway-infra namespace
cat <<EOF | kubectl apply -f -
apiVersion: gateway.networking.k8s.io/v1beta1
kind: ReferenceGrant
metadata:
  name: allow-checkout-route
  namespace: gateway-infra
spec:
  from:
  - group: gateway.networking.k8s.io
    kind: HTTPRoute
    namespace: checkout-app
  to:
  - group: gateway.networking.k8s.io
    kind: Gateway
    name: prod-gw
EOF

# Step 4: Verify route accepted
kubectl get httproute checkout-route -n checkout-app
```

- **Common Mistakes:** Creating the `ReferenceGrant` in the route's namespace (`checkout-app`) instead of the Gateway owner's namespace (`gateway-infra`). The resource granting permission must always reside in the namespace owning the target resource!
- **Troubleshooting:** Check controller logs: `kubectl logs -n gateway-infra -l app=gateway-controller`.
- **Cleanup Instructions:**

```
kubectl delete referencegrant allow-checkout-route -n gateway-infra
```